



Advanced Topics in Deep Learning
FINAL PROJECT

ADVANCED TIME SERIES

FORECASTING, OPTIMIZATION AND EXPLAINABILITY

António Cruz (140129), Cátia Brás (120093), Ricardo Kayseller (95813)

Table of Contents

1. Introduction and Project Goals	5
2. Environment, Reproducibility and Experimental Setup.....	7
2.1 Libraries Import	7
2.2 Configuration and Utils	7
3. Dataset and Problem Definition	9
3.1 Load Data	9
3.2 Initial Dataset Inspection	9
3.3 Datetime Parsing and Temporal Ordering	10
3.4 Hourly Resampling	10
3.5 Post-Resampling Quality Check	10
3.6 Handling Missing Values After Resampling	10
3.7 Forecasting Task and Variable Selection	10
4. Data Initial Preparation.....	12
4.1 Target Definition and Base Modeling DataFrame	12
4.2 Base Forecasting Data Overview.....	12
5. Feature Engineering	13
5.1 Cyclical Time Features.....	13
5.2 Wind-Derived Features	13
5.3 Final Feature Set for Modeling	14
6. Split, Scaling and Windowing	15
6.1 Temporal Train/Validation/Test Split	15
6.2 Feature Scaling.....	15
6.3 Target Index for Forecasting.....	15
6.4 Supervised Windowing.....	15
6.5 Windowed Data Sanity Check.....	16
7. Baseline Models	17
7.1 Persistence Baseline	17
7.2 GRU Baseline	18
7.3 Baseline Comparison.....	18
7.4 Reverse Scaling	19
7.5 Baseline Evaluation in Original Temperature Scale.....	19
7.6 Final Baseline Comparison.....	19
7.6 Forecast Visualization.....	19
8. Synthetic Data Generation with TimeGAN.....	21
8.1 Motivation and Experimental Role.....	21

8.2 TimeGAN Training Data Preparation	21
8.3 TimeGAN Model and Training Procedure	21
8.4 Synthetic Sequence Quality Assessment	24
9. Evolutionary Optimization of the Forecasting Pipeline	26
9.1 Motivation and Search Strategy	26
9.2 Search Space Definition	26
9.3 Evolutionary Representation and Constraints	27
9.4 Fitness Function	27
9.5 Evolutionary Search Execution	28
9.6 Best Configuration and Retraining.....	28
Then, we rebuild the best EA model to inspect its architecture and verify the parameter count.	28
9.7 Top Candidate Configurations	28
10. Retraining and Final Model Selection	30
10.1 Selection of Final Candidates	30
10.2 Retraining Procedure	31
10.3 Final Test Set Evaluation	32
10.4 Final Model Comparison	34
10.5 Final Model Selection.....	34
10.6 Robustness Across Random Seeds	34
11. Explainable AI	36
11.1 Global Explainability	36
11.2 Local Explainability	37
11.3 XAI Summary	39
11.4 XAI-Guided Feature Pruning.....	40
11.5 Effect of XAI-Guided Feature Pruning.....	41
12. Efficiency, Latency and Resource Analysis.....	42
12.1 Motivation	42
12.2 Models Selected for Efficiency Analysis.....	42
12.3 Training Time	42
12.4 Memory Usage (RAM / GPU).....	43
12.5 Trainable Parameters	43
12.6 Inference Latency	43
12.7 Efficiency Comparison	44
12.8 Final Efficiency Discussion	46
13. Comparative Discussion	48
Summary of Results	48

13.1 Baseline vs. EA-Optimized Performance	48
13.2 Impact of Evolutionary Optimization	48
13.3 Robustness Validation	49
13.4 Explainability Insights.....	49
13.5 Accuracy–Efficiency Trade-offs.....	49
13.6 Strengths and Limitations.....	49
14. Conclusion	51
14.1 Key Findings	51
14.2 XAI-Driven Model Refinement	51

1. Introduction and Project Goals

This project builds upon the Time Series Modelling mini project to develop a systematic, optimized, and well-analyzed forecasting system for air temperature prediction using the Jena Climate dataset. While the mini project focused on exploratory modelling and understanding different design choices, this Final Project elevates performance to a primary objective, achieved through systematic optimization and supported by rigorous analysis.

The core forecasting task is a **multivariate-input, univariate-output, multi-step prediction** problem given a window of historical meteorological measurements (temperature, pressure, humidity, wind speed, maximum wind speed, and wind direction), the system predicts the air temperature for the next 24 hours.

Our pipeline integrates the following advanced techniques:

- **Baseline Deep Learning Model (GRU):** A configurable multi-layer GRU architecture serves as the reference model against which all improvements are measured, incorporating regularization, gradient clipping, and modern training strategies (AdamW optimizer, Huber loss).
- **Evolutionary Optimization:** A genetic algorithm (GA) automatically searches over an end-to-end pipeline configuration space - jointly optimizing model hyperparameters, architectural choices, training settings, preprocessing strategy, regularization, and windowing parameters. With a population of 20 individuals over 15 generations (300 total evaluations), the EA discovers configurations that outperform the hand-tuned baseline while using fewer parameters.
- **Synthetic Data Generation (TimeGAN):** A Time-series Generative Adversarial Network generates realistic synthetic weather sequences from the training set, demonstrating the feasibility of data augmentation without information leakage.
- **Explainable AI (XAI):** Both global (permutation importance) and local (gradient saliency) explanation methods are applied. XAI insights are further used to guide **feature pruning**, removing redundant features to produce a simpler model with improved performance and robustness.
- **Efficiency and Resource Analysis:** Training time, inference latency, parameter counts, and memory usage (RAM and GPU) are profiled across model configurations, enabling an informed analysis of accuracy–efficiency trade-offs.

The remainder of this report is organized as follows:

- Section 2 describes our experimental setup and reproducibility measures.
- Section 3 presents the dataset and problem formulation.
- Sections 4–6 cover data preparation, feature engineering, and the windowing pipeline.

- Section 7 establishes baseline models.
- Section 8 details the Time GAN synthetic data generation.
- Section 9 presents the evolutionary optimization process.
- Section 10 covers final model retraining, selection, and robustness validation.
- Sections 11–12 present the explainability and efficiency analyses.
- Sections 13–14 provide the comparative discussion and conclusion.

2. Environment, Reproducibility and Experimental Setup

This section establishes the computational environment, library dependencies, random seed configuration, and project structure. Reproducibility is ensured through fixed seeds, explicit dependency management, and modular code organization.

2.1 Libraries Import

```
import os
import gc
import sys
import json
import random
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import optuna
import psutil
import subprocess
```

We verify that TensorFlow detects the available GPU and configure memory growth to prevent out-of-memory errors during the evolutionary search, which trains many models sequentially.

2.2 Configuration and Utils

We also set a global random seed (SEED = 42) applied consistently to Python, NumPy, and TensorFlow via `set_global_seed()` to ensure reproducibility. GPU memory growth is enabled via `enable_gpu_memory_growth()`, and device information is printed to confirm the hardware configuration.

```
SEED = 42

os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Set up the project root path and import custom utility functions.

```
PROJECT_ROOT = Path.cwd().parent
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

print(PROJECT_ROOT)
/home/ricarl/taap_p4
```

We import our custom utility functions and verify the environment setup (seed, GPU memory growth, device info).

```
from src.utils.env import set_global_seed, enable_gpu_memory_growth, get_device_info

SEED = 42
set_global_seed(SEED)
enable_gpu_memory_growth()

print(get_device_info())
{'tensorflow_version': '2.21.0', 'num_gpus': 1, 'gpus': ['/physical_device:GPU:0'], 'num_cpus': 1, 'cpus': ['/physical_device:CPU:0']}
```


3. Dataset and Problem Definition

The Jena Climate dataset serves as the foundation for all experiments. This section covers data loading, quality assurance, temporal resampling to hourly resolution, and the formal definition of the forecasting task and variable selection.

3.1 Load Data

The Jena Climate dataset is loaded from a local CSV file. The dataset contains meteorological measurements collected at a weather station in Jena, Germany, between January 2009 and December 2016, with an original sampling frequency of 10 minutes across 15 variables.

3.2 Initial Dataset Inspection

Upon loading, we inspect the dataset shape, column names, data types, and general structure. This provides a first look at the raw data before any transformations are applied.

Table 1 - First rows of the raw Jena Climate dataset (10-minute sampling, 15 variables).

#	Column	Non-Null Count	Dtype
0	Date Time	420551 non-null	str
1	p (mbar)	420551 non-null	float64
2	T (degC)	420551 non-null	float64
3	Tpot (K)	420551 non-null	float64
4	Tdew (degC)	420551 non-null	float64
5	rh (%)	420551 non-null	float64
6	VPmax (mbar)	420551 non-null	float64
7	VPact (mbar)	420551 non-null	float64
8	VPdef (mbar)	420551 non-null	float64
9	sh (g/kg)	420551 non-null	float64
10	H2OC (mmol/mol)	420551 non-null	float64
11	rho (g/m**3)	420551 non-null	float64
12	wv (m/s)	420551 non-null	float64
13	max. wv (m/s)	420551 non-null	float64
14	wd (deg)	420551 non-null	float64

dtypes: float64(14), str(1)
memory usage: 48.1 MB

After loading the raw dataset, we perform quality checks to ensure data integrity before any transformations. Specifically, we verify the presence of duplicated rows and inspect all columns for missing values. Any duplicated observations are removed to avoid biasing the resampling step that follows - duplicate timestamps can arise from logging artifacts and must be eliminated to ensure a consistent temporal index.

3.3 Datetime Parsing and Temporal Ordering

The Date Time column is parsed into proper datetime objects and the dataframe is sorted chronologically. We verify the date range and inspect the time delta distribution to detect any irregularities in the 10-minute sampling frequency before resampling.

Table 2 - Dataset date range and first rows after datetime parsing and temporal ordering.

Date Time	
0	2009-01-01 00:10:00
1	2009-01-01 00:20:00
2	2009-01-01 00:30:00
3	2009-01-01 00:40:00
4	2009-01-01 00:50:00

We also check the time delta distribution to identify sampling irregularities and inspect the distribution of time deltas between consecutive observations to detect any sampling irregularities.

3.4 Hourly Resampling

The raw 10-minute resolution is resampled to **1-hour intervals** using `resample("1h").mean()`, aggregating all observations within each hourly bin into their arithmetic mean. This reduces the dataset by a factor of 6 while preserving the dominant meteorological patterns relevant for 24-hour forecasting. Mean aggregation was chosen over alternatives (e.g., first/last, median) because it preserves the central tendency while smoothing sub-hourly noise.

3.5 Post-Resampling Quality Check

After resampling, we re-validate the dataset to ensure no artifacts were introduced. We verify that no duplicate hourly timestamps exist, that the time delta between consecutive rows is consistently 1 hour, and that any missing values from incomplete hourly bins are identified and quantified. This step is critical because the sliding window approach assumes a regular, gap-free time series.

3.6 Handling Missing Values After Resampling

Any rows containing NaN values after resampling are dropped. Since the original dataset has near-complete 10-minute coverage across 2009–2016, the number of affected rows is minimal (88 out of ~70,000) and does not introduce meaningful temporal gaps. Dropping is preferred over imputation because the affected bins are at dataset boundaries and isolated gaps where interpolation would risk introducing artificial patterns.

3.7 Forecasting Task and Variable Selection

Following the project specification, we retain 6 meteorological variables plus the datetime reference: T (degC) (air temperature, the target), p (mbar) (atmospheric pressure), rh (%)

(relative humidity), wv (m/s) (wind speed), $\max. wv$ (m/s) (maximum wind speed), and wd (deg) (wind direction). All other variables from the original 15-column dataset are excluded.

This defines a **multivariate-input, univariate-output, multi-step** forecasting problem: the model receives a multi-day window of multi-sensor weather data and must predict the air temperature trajectory for the next 24 hours. The multivariate input ensures the model can leverage cross-variable dependencies - for instance, the relationship between falling pressure and subsequent temperature changes driven by weather fronts.

Table 3 - Selected variables for the forecasting task (6 meteorological features + datetime).

	Date Time	T (degC)	p (mbar)	rh (%)	wv (m/s)	max. wv (m/s)	wd (deg)
0	2009-01-01 00:00:00	-8.304000	996.528000	93.780000	0.520000	1.002000	174.460000
1	2009-01-01 01:00:00	-8.065000	996.525000	93.933333	0.316667	0.711667	172.416667
2	2009-01-01 02:00:00	-8.763333	996.745000	93.533333	0.248333	0.606667	196.816667
3	2009-01-01 03:00:00	-8.896667	996.986667	93.200000	0.176667	0.606667	157.083333
4	2009-01-01 04:00:00	-9.348333	997.158333	92.383333	0.290000	0.670000	150.093333

4. Data Initial Preparation

Before applying feature engineering, we establish the core modelling dataframe by defining the target variable and verifying the dataset structure. This section bridges the raw data cleaning (Section 3) with the feature derivation pipeline (Section 5), ensuring a clear separation of concerns.

4.1 Target Definition and Base Modeling DataFrame

The target variable is defined as T (degC) - air temperature in degrees Celsius. This is the only quantity the model must forecast in the output window; all other variables serve as exogenous inputs.

The Date Time column serves as the temporal reference for feature engineering (extracting hour-of-day and day-of-year), but is not included as a direct model input since neural networks cannot interpret raw datetime objects. Instead, its temporal information is encoded as cyclic features in Section 5.

The remaining 5 meteorological variables - atmospheric pressure, relative humidity, wind speed, maximum wind speed, and wind direction - form the covariate set that provides the atmospheric context for the temperature forecast.

4.2 Base Forecasting Data Overview

We inspect the prepared dataframe to confirm shape, date range, and column integrity before proceeding to feature engineering. The dataset at this stage contains approximately 70,000 hourly observations spanning from January 2009 to January 2017, with 7 columns (datetime + 6 meteorological variables).

This verification step ensures that no data was inadvertently lost during the resampling and cleaning pipeline of Section 3 and establishes the baseline from which all derived features will be computed.

5. Feature Engineering

Two families of derived features are introduced to enrich the input representation: **cyclical temporal encodings** and **wind-derived features**.

The goal is to provide the neural network with representations that respect the physical structure of the data - cyclic quantities are encoded cyclically, and vector quantities are decomposed into Cartesian components.

All feature engineering is implemented in `src.features.engineering` for reproducibility.

5.1 Cyclical Time Features

Hour-of-day and day-of-year carry strong periodic signals (diurnal and seasonal cycles). Encoding them as raw integers would introduce artificial discontinuities (e.g., hour 23 far from hour 0). We apply a sine-cosine encoding that maps each cyclic variable to a point on the unit circle, ensuring smooth transitions at boundaries:

- `hour_sin`, `hour_cos`
- diurnal cycle (period = 24h)
- `doy_sin`, `doy_cos`
- seasonal cycle (period = 365.25 days)

Table 4 - Cyclical temporal features derived from hour-of-day and day-of-year.

	Date Time	hour	dayofyear	hour_sin	hour_cos	doy_sin	doy_cos
0	2009-01-01 00:00:00	0	1	0.000000	1.000000	0.017202	0.999852
1	2009-01-01 01:00:00	1	1	0.258819	0.965926	0.017202	0.999852
2	2009-01-01 02:00:00	2	1	0.500000	0.866025	0.017202	0.999852
3	2009-01-01 03:00:00	3	1	0.707107	0.707107	0.017202	0.999852
4	2009-01-01 04:00:00	4	1	0.866025	0.500000	0.017202	0.999852

5.2 Wind-Derived Features

Wind direction in degrees is inherently circular and difficult for neural networks to interpret directly.

We derive six features:

- `wd_sin`, `wd_cos` - cyclic encoding of wind direction
- `wx`, `wy` - Cartesian decomposition of wind vector (speed $\times$ cos/sin of direction), replacing the polar representation with a form neural networks process more naturally
- `wind_gap` - difference between max and sustained wind speed (gust intensity)
- `gust_ratio` - ratio of max to sustained wind speed (relative gust strength, with $\varepsilon = 10^{-6}$ for numerical stability)

Table 5 - Wind-derived features: cyclic encoding, Cartesian components, gust metrics.

Date Time	wv (m/s)	max. wv (m/s)	wd (deg)	wd_sin	wd_cos	wx	wy	wind_gap	gust_ratio
2009-01-01 00:00:00	0.520000	1.002000	174.460000	0.096541	-0.995329	-0.517571	0.050201	0.482000	1.926919
2009-01-01 01:00:00	0.316667	0.711667	172.416667	0.131968	-0.991254	-0.313897	0.041790	0.395000	2.247361
2009-01-01 02:00:00	0.248333	0.606667	196.816667	-0.289310	-0.957235	-0.237713	-0.071845	0.358333	2.442943
2009-01-01 03:00:00	0.176667	0.606667	157.083333	0.389392	-0.921072	-0.162723	0.068793	0.430000	3.433943
2009-01-01 04:00:00	0.290000	0.670000	150.093333	0.498589	-0.866839	-0.251383	0.144591	0.380000	2.310337

5.3 Final Feature Set for Modeling

The complete input feature vector contains **16 dimensions**: 6 original meteorological variables + 4 cyclical temporal encodings + 6 wind-derived features. This feature set is used consistently across all experiments - baseline, evolutionary optimization, and synthetic data generation.

```
print("Number of modeling features:", len(final_feature_cols))
print(final_feature_cols)
Number of modeling features: 16
['T (degC)', 'p (mbar)', 'rh (%)', 'wv (m/s)', 'max. wv (m/s)', 'wd (deg)', 'hour_sin', 'hour_cos', 'doy_sin', 'doy_cos', 'wd_sin', 'wd_cos', 'wx', 'wy', 'wind_gap', 'gust_ratio']
```

The XAI analysis in Section 11 later evaluates which of these 16 features contribute most, leading to a pruned 11-feature variant.

6. Split, Scaling and Windowing

This section completes the data preparation pipeline by splitting the dataset temporally, normalizing features, and creating the supervised learning windows that feed into all models.

6.1 Temporal Train/Validation/Test Split

The dataset is split chronologically (70% / 15% / 15%) with **no shuffling** - the split respects temporal order to prevent data leakage. The training set covers the earliest years, followed by validation, with the most recent data reserved for testing. This mirrors a realistic deployment scenario where the model is trained on historical data and evaluated on future observations.

6.2 Feature Scaling

All input features are normalized using **StandardScaler** (zero mean, unit variance) as the baseline strategy. The scaler is fit exclusively on the training set and applied via transform to validation and test sets, preventing information leakage.

The implementation of evolutionary optimization also searches over the scaler type (standard, robust, minmax), allowing the GA to discover whether a different normalization strategy benefits specific model configurations.

We apply the selected scaler to all splits (fit on train, transform on val/test), and apply the scaler to the training, validation, and test feature matrices. The scaler is fit only on the training data.

6.3 Target Index for Forecasting

We identify the position of T (degC) within the 16-feature vector. This index is used during windowing to extract only the temperature column for the output sequence **y**, while keeping all 16 features in the input tensor **X**. This separation makes the problem multivariate-input but univariate-output.

6.4 Supervised Windowing

The scaled data is converted into supervised learning samples using a sliding window:

- **Input (X):** (LOOKBACK, 16) - consecutive hourly observations across all features
- **Output (y):** (HORIZON,) = 24 hourly temperature values to predict

The default lookback is 120 hours (5 days). The evolutionary optimization in Section 9 also explores alternative lookback lengths as part of the search space.

We create supervised windows for all splits using the defined lookback and horizon, and create supervised windows for train, validation, and test splits using the defined lookback (120h) and horizon (24h).

6.5 Windowed Data Sanity Check

We verify the shapes of all windowed arrays to confirm correct dimensionality and consistency. The expected shapes are: input (N, LOOKBACK, 16) and output (N, 24). This verification catches common off-by-one errors in window construction that would silently corrupt the training process.

```
print("Input shape:", X_train.shape[1:])
print("Forecast horizon:", y_train.shape[1])
print("Number of input features:", X_train.shape[2])

print("\nExample input window shape:", X_train[0].shape)
print("Example target shape:", y_train[0].shape)
print("First 5 target values (scaled):", y_train[0][:5])
Input shape: (120, 16)
Forecast horizon: 24
Number of input features: 16

Example input window shape: (120, 16)
Example target shape: (24,)
First 5 target values (scaled): [-2.55218147 -2.61156348 -2.64453206 -2.68540539
-2.73726824]
```

7. Baseline Models

Two baseline models are established as reference points. The **persistence baseline** provides the lowest bar - the performance of a no-skill forecast that any learned model must surpass. The **GRU baseline** represents a carefully designed deep learning model with modern training practices, serving as the benchmark against which the evolutionary optimization is measured.

7.1 Persistence Baseline

The simplest possible forecast: repeat the last observed temperature value across the entire 24-hour horizon. This naive baseline provides a scale-aware reference - any learned model that fails to beat persistence adds no value. We also build a simple 1-layer GRU as a quick sanity check before the official configurable baseline.

Compute scaled metrics for the persistence forecast on the test set:

```
print("Persistence prediction shape:", y_pred_persistence.shape)
print("First prediction:", y_pred_persistence[0][:5])
Persistence prediction shape: (10364, 24)
First prediction: [-0.04174932 -0.04174932 -0.04174932 -0.04174932 -0.04174932]
```

Compute persistence baseline metrics on the test set:

```
print("Persistence MAE (scaled):", persistence_mae_scaled)
print("Persistence RMSE (scaled):", persistence_rmse_scaled)
Persistence MAE (scaled): 0.3636531434000367
Persistence RMSE (scaled): 0.4921206134926399
```

Before building the official configurable baseline, we construct a simple 1-layer GRU (64 units) as a quick sanity check. This verifies that the full pipeline - from windowed data through model training to test evaluation - works correctly end-to-end. The simple model's results are not used for comparison; they serve only as a development checkpoint.

After training the simple GRU with early stopping on validation loss, we generate and inspect test set predictions from the simple GRU and Evaluate the simple GRU on scaled metrics to compare against persistence.

```
print("Prediction shape:", y_pred_gru.shape)
print("First prediction:", y_pred_gru[0][:5])
Prediction shape: (10364, 24)
First prediction: [-0.13829982 -0.19688366 -0.15950276 -0.10881609 -0.00536779]
```

```
print("GRU MAE (scaled):", gru_mae_scaled)
print("GRU RMSE (scaled):", gru_rmse_scaled)
```

```
GRU MAE (scaled): 0.19162778158532956
GRU RMSE (scaled): 0.25370004555151926
```

7.2 GRU Baseline

The official baseline uses `build_gru_model()` from `src.models.gru` - a configurable 2-layer GRU (96→64 units) with AdamW optimizer, Huber loss ($\delta=1.0$), L2 regularization, gradient clipping, and an intermediate dense layer (256 units). Training uses early stopping (patience=6) and learning rate reduction (patience=3, factor=0.5).

We then define the official baseline configuration and build the model. The official GRU baseline is built using `build_gru_model()` from `src.models.gru`, which supports configurable depth, width, regularization, optimizer, and loss function. The baseline configuration was manually tuned based on prior experimentation in the mini-project, providing a strong reference point for the evolutionary search.

The official baseline is trained with the full 60-epoch budget. Training uses early stopping (patience=6, restoring best weights) and learning rate reduction on plateau (patience=3, factor=0.5), ensuring the model converges without overfitting.

We generate test predictions from the official baseline, inspect the output and compute scaled MAE and RMSE for the official GRU baseline on the test set:

```
print("Prediction shape:", y_pred_gru.shape)
print("First prediction:", y_pred_gru[0][:12])
Prediction shape: (10364, 24)
First prediction: [-0.12780994 -0.14077106 -0.10512683 -0.07985469 -0.02349082
 0.0554703
 0.1347603  0.26358065  0.2870692  0.31509727  0.29663756  0.222746  ]
```

```
print("GRU Baseline MAE (scaled):", gru_mae_scaled)
print("GRU Baseline RMSE (scaled):", gru_rmse_scaled)
GRU Baseline MAE (scaled): 0.19262296176021332
GRU Baseline RMSE (scaled): 0.2555149343830984
```

7.3 Baseline Comparison

Side-by-side comparison of persistence and GRU baseline on scaled metrics (MAE and RMSE on standardized data). Scaled metrics are useful for monitoring training but lack physical interpretation - for this reason we also compute original-scale metrics below.

Table 6 - Persistence vs. GRU baseline comparison on scaled metrics (test set).

	Model	MAE_scaled	RMSE_scaled
0	Persistence	0.363653	0.492121
1	GRU Baseline Official	0.192623	0.255515

7.4 Reverse Scaling

Predictions and ground truth are inverse transformed from normalized space back to degrees Celsius using the training set's scaler statistics (mean and standard deviation of the temperature column). This allows evaluation in physically meaningful units.

```
print("y_test_inv shape:", y_test_inv.shape)
print("y_pred_gru_inv shape:", y_pred_gru_inv.shape)
y_test_inv shape: (10364, 24)
y_pred_gru_inv shape: (10364, 24)
```

7.5 Baseline Evaluation in Original Temperature Scale

We compute MAE and RMSE in degrees Celsius on the inverse-scaled predictions. These original-scale metrics are the primary comparison basis throughout the report - an MAE of 1.65°C means the model's 24-hour forecasts are off by an average of 1.65 degrees. This is the metric used as the evolutionary fitness function and the final evaluation criterion.

```
print("GRU Baseline MAE (°C):", gru_mae)
print("GRU Baseline RMSE (°C):", gru_rmse)
GRU Baseline MAE (°C): 1.6651472981896625
GRU Baseline RMSE (°C): 2.208822866947564
```

7.6 Final Baseline Comparison

Combined comparison table with both scaled and original-scale (°C) metrics. The GRU baseline achieves a substantial improvement over persistence, confirming that the recurrent architecture captures temporal dependencies in the meteorological data.

These results establish the performance ceiling that the evolutionary optimization will attempt to surpass.

Table 7 - Final baseline comparison with both scaled and original-scale (°C) metrics.

	Model	MAE_scaled	RMSE_scaled	MAE_degC	RMSE_degC
0	Persistence	0.363653	0.492121	3.143634	4.254183
1	GRU Baseline Official	0.192623	0.255515	1.665147	2.208823

7.6 Forecast Visualization

Visual comparison of a single 24-hour forecast window: ground truth vs. persistence vs. GRU baseline predictions in °C. This provides an intuitive sense of how the models track the temperature trajectory.

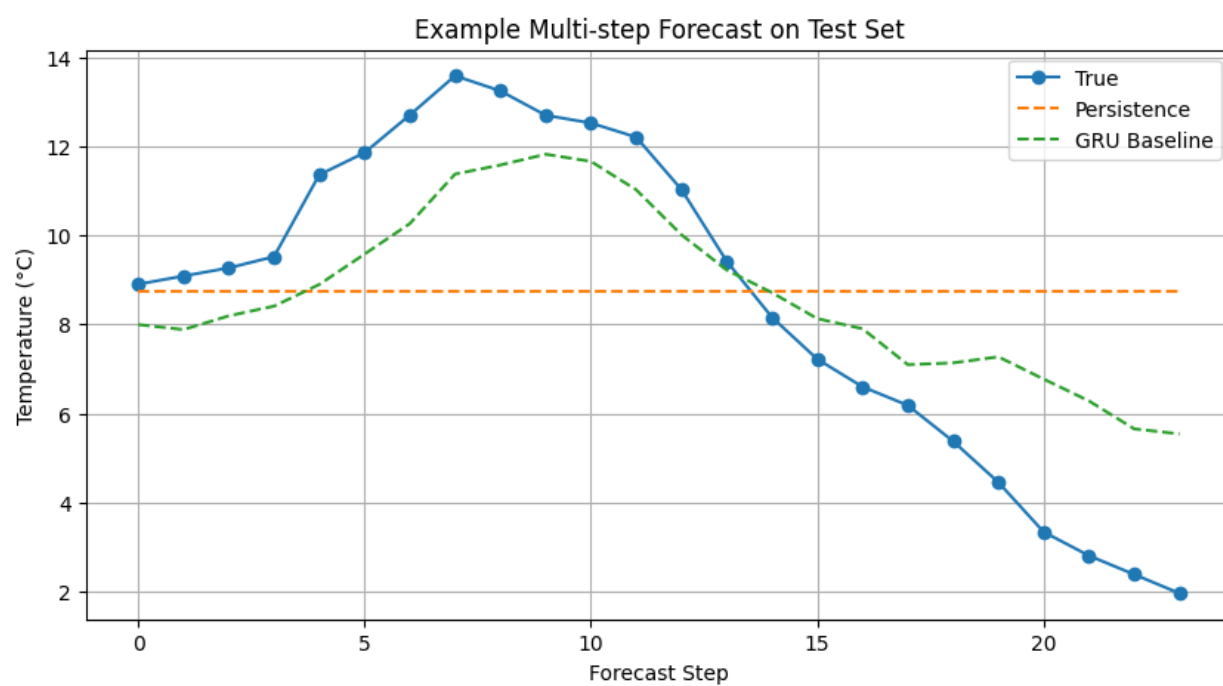


Figure 1 - 24-hour forecast comparison: ground truth vs. persistence vs. GRU baseline (°C).

8. Synthetic Data Generation with TimeGAN

This section explores generative approaches for creating realistic synthetic weather sequences, with the goal of demonstrating data augmentation feasibility without information leakage.

8.1 Motivation and Experimental Role

The goal of the TimeGAN component is to generate realistic synthetic multivariate weather sequences using only the training split, avoiding any form of data leakage.

These synthetic sequences will later be used to augment the training data and assess whether data augmentation improves forecasting performance, robustness, or generalization.

To keep the setup consistent with the forecasting task, the synthetic generator is trained on multivariate sequences of length `LOOKBACK + HORIZON`, so that each generated sequence can later be split into: - an input window of length `LOOKBACK` - a target forecasting horizon of length `HORIZON`

The impact of synthetic augmentation will be evaluated by comparing forecasting models trained with: 1. real data only 2. real data plus synthetic data

8.2 TimeGAN Training Data Preparation

Training data is converted into fixed-length sequences of `LOOKBACK + HORIZON = 144` time steps, matching the forecasting window. This ensures generated sequences can be directly split into input-output pairs. Only the **training set** is used - validation and test data are never exposed to the generative model, preventing data leakage.

```
print("TimeGAN sequence length:", TIMEGAN_SEQ_LEN)
print("TimeGAN training sequences shape:", timegan_train_sequences.shape)
TimeGAN sequence length: 144
TimeGAN training sequences shape: (48885, 144, 16)
```

8.3 TimeGAN Model and Training Procedure

TimeGAN (Yoon et al., 2019) consists of five GRU-based sub-networks: **Embedder** (data → latent), **Recovery** (latent → data), **Generator** (noise → latent), **Supervisor** (temporal dynamics in latent space), and **Discriminator** (real vs. synthetic). Each uses 3 GRU layers with hidden dimension 24. Training follows a three-phase curriculum: autoencoder pretraining, supervisor pretraining, and adversarial training.

8.3.1 Autoencoder Pretraining

Phase 1: The embedder and recovery networks are jointly trained to minimize reconstruction error (MSE) on real sequences. This establishes a meaningful latent space before adversarial training begins.

```

autoencoder_history = timegan.pretrain_autoencoder(
    sequences=timegan_train_sequences,
    epochs=TIMEGAN_CONFIG["ae_epochs"],
    batch_size=TIMEGAN_CONFIG["batch_size"],
    verbose=1,
)

```

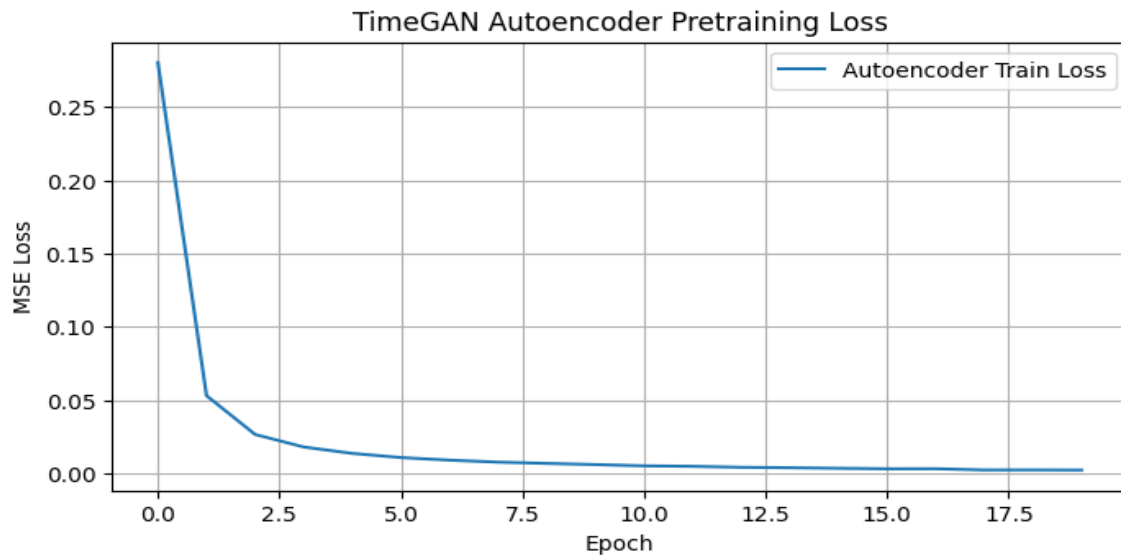


Figure 2 - TimeGAN autoencoder pretraining loss (MSE) over 20 epochs.

8.3.2 Supervisor Pretraining

Phase 2: The supervisor network learns to predict the next latent time step from the current one, using embeddings from the (now frozen) embedder. This captures temporal dynamics in latent space.

```

supervisor_history = timegan.pretrain_supervisor(
    sequences=timegan_train_sequences,
    epochs=TIMEGAN_CONFIG["sup_epochs"],
    batch_size=TIMEGAN_CONFIG["batch_size"],
    verbose=1,
)

```

8.3.3 Pretraining Loss Curves

Combined visualization of autoencoder and supervisor pretraining losses to verify both networks converged before adversarial training.

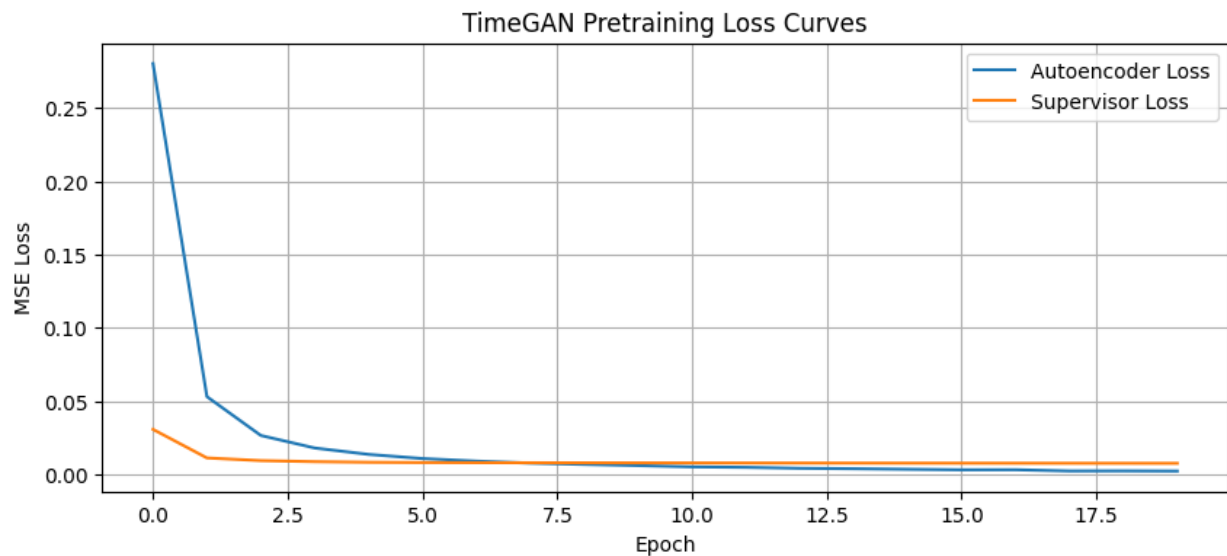


Figure 3 - Combined pretraining loss curves for autoencoder and supervisor networks.

8.3.4 Adversarial Training

Phase 3: Generator and discriminator are trained adversarially. The total generator loss combines three objectives: adversarial (fool discriminator), supervised (temporal coherence in latent space, weighted $\times 100$), and reconstruction (data-space fidelity, weighted by $\gamma=1.0$). A discriminator update threshold prevents it from dominating too early.

```
adversarial_history = timegan.fit(
    sequences=timegan_train_sequences,
    epochs=TIMEGAN_CONFIG["adv_epochs"],
    batch_size=TIMEGAN_CONFIG["batch_size"],
    verbose=1,
)
```

8.3.5 Adversarial Training Loss Curves

Visualization of discriminator and generator losses across adversarial epochs. A well-trained TimeGAN shows the two networks reaching dynamic equilibrium rather than one dominating.

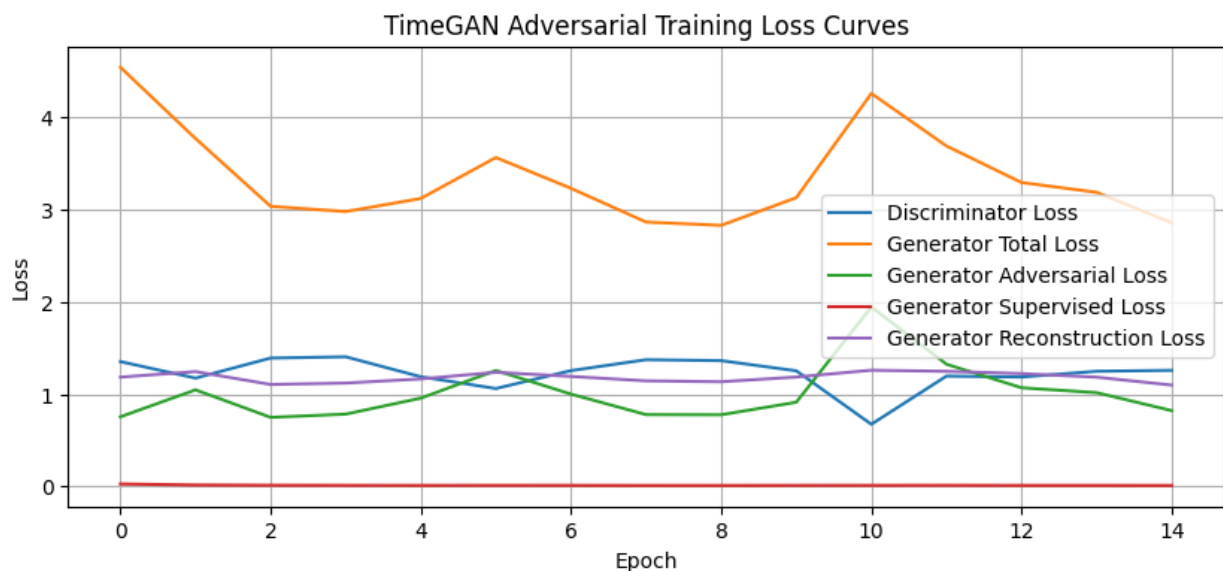


Figure 4 - TimeGAN adversarial training loss curves (discriminator, generator, supervised, reconstruction).

8.4 Synthetic Sequence Quality Assessment

Before considering synthetic data for augmentation, we assess whether the TimeGAN produces sequences that are statistically and temporally faithful to the real training data through reconstruction checks and visual comparisons.

8.4.1 Autoencoder Reconstruction Check

We pass real sequences through the embedder → recovery path to verify the latent space captures meaningful structure. Low reconstruction MSE indicates the autoencoder learned a faithful representation.

```
print("Reconstructed batch shape:", reconstructed_sequences.shape)
print("Reconstruction MSE on sample batch:", reconstruction_mse)
Reconstructed batch shape: (256, 144, 16)
Reconstruction MSE on sample batch: 0.0011650081324751588
```

8.4.2 Real vs Reconstructed Sequence

Visual comparison of a real temperature sequence against its autoencoder reconstruction. Close overlap confirms the latent space preserves temporal structure.

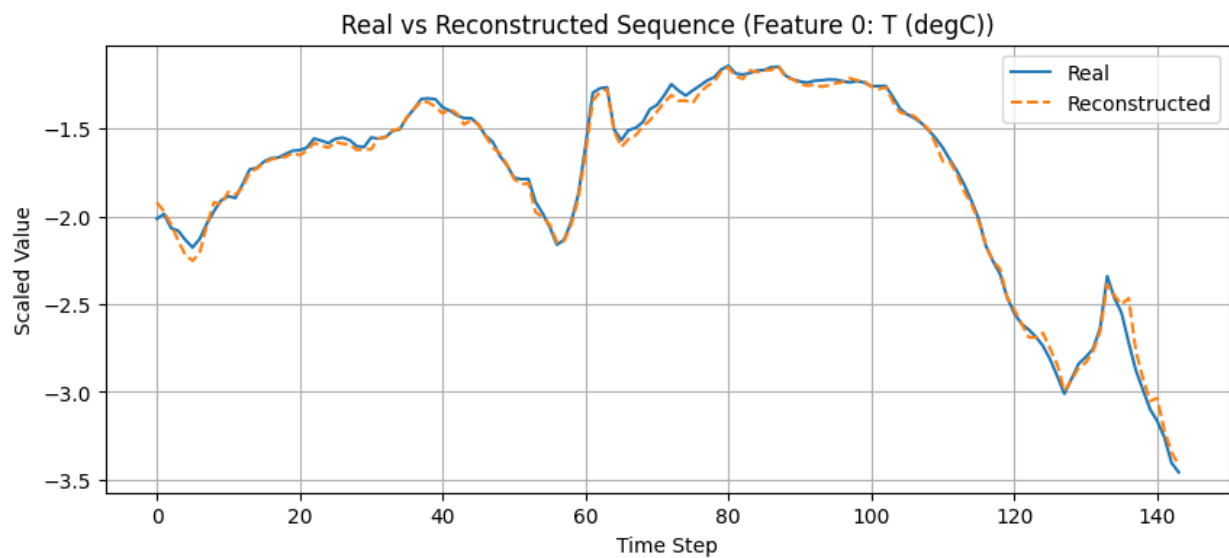


Figure 5 - Real vs. reconstructed temperature sequence through the TimeGAN autoencoder.

8.4.3 Synthetic Sequence Preview

Generate a small batch of synthetic sequences and inspect their shape and value range to verify the generator produces outputs within plausible bounds.

```
Synthetic preview shape: (8, 144, 16)
Synthetic preview min: -1.002639
Synthetic preview max: 0.7755798
```

8.4.4 Real vs Synthetic Sequence

Visual comparison of a real training sequence against a generated synthetic sequence for the temperature feature, providing a qualitative check of temporal dynamics and value distributions.

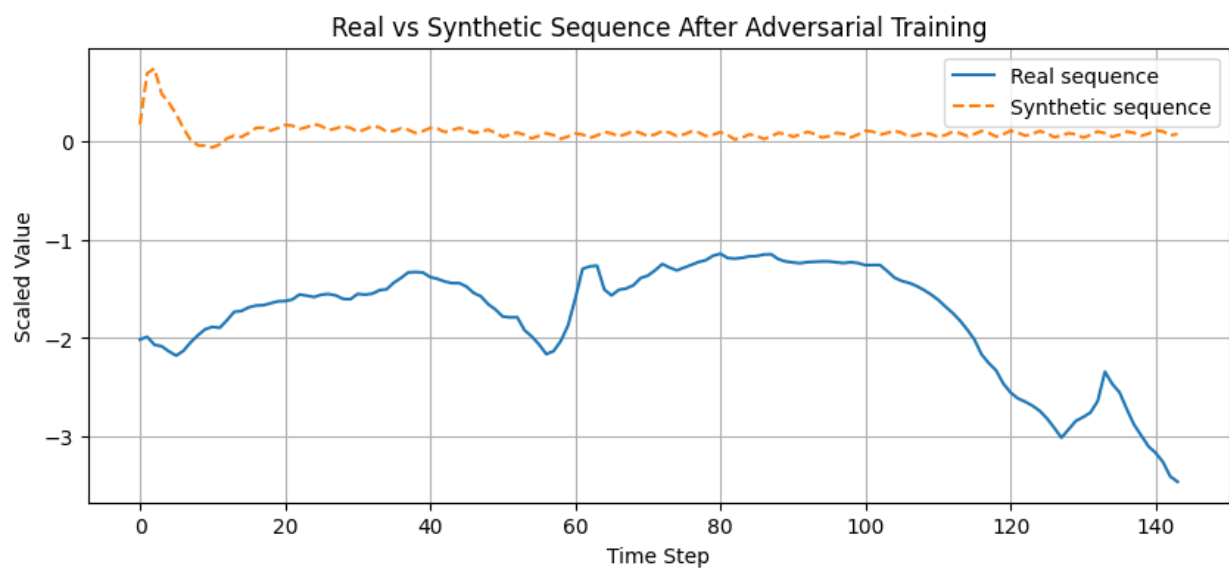


Figure 6 - Real training sequence vs. TimeGAN-generated synthetic sequence (temperature feature).

9. Evolutionary Optimization of the Forecasting Pipeline

9.1 Motivation and Search Strategy

The evolutionary optimization stage aims to improve the forecasting pipeline beyond the fixed GRU baseline by exploring alternative architectural and training configurations.

Instead of relying on a single manually selected model or Bayesian optimization from previous work, this stage uses a genetic search strategy based on:

- population initialization
- tournament selection
- crossover
- mutation
- elitism

Each candidate solution represents a full forecasting configuration, including GRU architecture, optimizer, loss, regularization, batch size, and scaling method. The fitness of each candidate is defined by validation forecasting performance on the scaled target series.

9.2 Search Space Definition

The search space spans **16 hyperparameters** across five categories:

- **Architecture:** number of GRU layers (1–2), units per layer (64–128 / 32–96 / 16–64), optional dense layer (0–256 units), dense activation (ReLU, GELU, ELU, LeakyReLU)
- **Regularization:** dropout (0.0–0.3), L2 ($0-10^{-4}$), Gaussian noise (0–0.05), gradient clipping (0.5–5.0)
- **Training:** optimizer (Adam, AdamW), weight decay ($0-10^{-4}$), learning rate ($10^{-4}-2\times 10^{-3}$), loss (MSE, MAE, Huber $\delta=1$, Huber $\delta=2$), batch size (128–512)
- **Preprocessing:** scaler type (standard, robust, minmax)
- **Windowing:** lookback length (explored as part of the search)

The search space was refined from an initial broader range by removing 3-layer architectures and capping unit sizes, concentrating the search on configurations most likely to be effective for this dataset.

9.2.1 Windowing Search Extension

Unlike the mini-project, the windowing configuration was not treated as fixed. In addition to architecture and training hyperparameters, the evolutionary search was extended to explore alternative lookback lengths while keeping the 24-hour forecast horizon fixed. This allowed the optimization process to jointly search over the forecasting model and the temporal context length used as input.

9.3 Evolutionary Representation and Constraints

Each individual is a dictionary mapping gene names to values from the search space. Constraints enforce valid architectures: monotonically decreasing layer widths ($\text{units}_2 \leq \text{units}_1$, $\text{units}_3 \leq \text{units}_2$) and optimizer–weight-decay consistency ($\text{weight_decay} = 0$ for standard Adam). These constraints are applied after initialization, crossover, and mutation.

```
example_genotype = sample_genotype()
example_genotype
{'n_layers': 2,
 'units1': 128,
 'units2': 64,
 'units3': 64,
 'dropout': 0.0,
 'l2': 0.0,
 'dense_units': 64,
 'dense_activation': 'elu',
 'learning_rate': 0.001,
 'batch_size': 128,
 'clipnorm': 0.5,
 'optimizer_name': 'adamw',
 'weight_decay': 1e-05,
 'loss_name': 'huber2',
 'gaussian_noise_std': 0.01,
 'scaler_name': 'standard',
 'lookback': 120}
```

9.4 Fitness Function

The fitness function encapsulates the complete forecasting pipeline, from data scaling through model training to validation prediction, within a single callable that returns a scalar fitness score. The fitness of everyone is defined as the validation MAE on the scaled forecasting target.

For each candidate configuration, the pipeline performs:

1. feature scaling using the candidate scaler
2. supervised window generation
3. GRU model construction
4. model training on the training split
5. prediction on the validation split
6. computation of validation MAE in scaled space

The optimization objective is to minimize this validation forecasting error.

9.5 Evolutionary Search Execution

The GA runs with **population size 20** over **15 generations** (300 total evaluations), using tournament selection ($k=3$), uniform crossover, per-gene mutation (rate=0.2), and elitism (top individuals preserved). Each individual is trained for up to 20 epochs with early stopping and LR reduction. The fitness metric is MAE in °C on the validation set - using original-scale error ensures invariance to the scaler choice.

Overfitting mitigation: Fitness is computed on the validation set only. The test set remains untouched during the search. Robustness is validated in Section 10.6 by re-evaluating the best configuration across multiple random seeds.

9.6 Best Configuration and Retraining

We extract the best configuration found by the evolutionary search - the individual with the lowest validation MAE across all generations. This configuration encodes the full pipeline: scaler type, GRU architecture, regularization, optimizer settings, loss function, and lookback length.

```
best_cfg = best_result["cfg"]
best_cfg
{'n_layers': 2,
 'units1': 64,
 'units2': 64,
 'units3': 32,
 'dropout': 0.0,
 'l2': 1e-05,
 'dense_units': 256,
 'dense_activation': 'relu',
 'learning_rate': 0.0003,
 'batch_size': 256,
 'clipnorm': 5.0,
 'optimizer_name': 'adamw',
 'weight_decay': 0.0,
 'loss_name': 'mae',
 'gaussian_noise_std': 0.0,
 'scaler_name': 'robust',
```

Then, we rebuild the best EA model to inspect its architecture and verify the parameter count.

9.7 Top Candidate Configurations

We analyze the top-ranked individuals from the final EA generation to understand the diversity of solutions discovered. This helps assess whether the EA converged to a single region of the search space or explored multiple competitive alternatives.

Table 8 - Top candidate configurations discovered across all EA generations.

Parameter	Candidate 1	Candidate 2	Candidate 3	Candidate 4	Candidate 5
n_layers	2	2	2	2	2
units1/units2/units3	64 / 64 / 32	96 / 96 / 64	128 / 128 / 32	64 / 64 / 32	128 / 64 / 64
dropout	0.0	0.0	0.0	0.0	0.0
L2	0.000010	0.000001	0.000100	0.000001	0.000001
dense_units	256	256	256	256	256
dense_activation	relu	relu	relu	relu	relu
learning_rate	0.0003	0.0003	0.0003	0.0003	0.0002
batch_size	256	256	256	256	128
clipnorm	5.0	5.0	5.0	5.0	5.0
optimizer_name	adamw	adam	adam	adam	adam
weight_decay	0.0	0.0	0.0	0.0	0.0
loss_name	mae	mae	mae	mae	mae
gaussian_noise_std	0.0	0.0	0.0	0.0	0.0
scaler_name	robust	robust	robust	robust	robust
lookback	144	144	144	144	144
fitness_mae_degC	1.635730	1.635795	1.636511	1.637370	1.638159
rmse_degC	2.237388	2.229955	2.242214	2.239683	2.237043
mae_scaled	0.132586	0.132592	0.132650	0.132719	0.132783
rmse_scaled	0.181355	0.180752	0.181746	0.181541	0.181327

10. Retraining and Final Model Selection

Results - Sections 10–12 present the final evaluation results, explainability analysis, and efficiency profiling.

The best EA configuration was identified using a reduced training budget (20 epochs per individual). To obtain a fair final comparison, we now retrain both the baseline and the EA-optimized model with the full training budget (60 epochs), evaluate on the held-out test set, and validate robustness across multiple random seeds.

10.1 Selection of Final Candidates

We select two candidates for final evaluation: the hand-tuned GRU baseline and the best EA-discovered configuration. Each candidate carries its own complete pipeline specification - architecture, optimizer, loss, scaler, and lookahead - ensuring a fully controlled comparison.

Table 9 - Final candidate configurations: GRU baseline vs. best EA-discovered pipeline.

Parameter	GRU Baseline Official	Best Evolutionary GRU
n_layers	2	2
units1	96	64
units2	64	64
units3	96	32
dropout	0.0	0.0
l2	0.000001	0.000010
dense_units	256	256
dense_activation	relu	relu
learning_rate	0.0002	0.0003
clipnorm	2.0	5.0
optimizer_name	adamw	adamw
weight_decay	0.000001	0.000000
loss_name	huber1	mae
gaussian_noise_std	0.0	0.0
batch_size	128	256
scaler_name	standard	robust
lookback	120	144.0

10.2 Retraining Procedure

Both candidates are retrained from scratch with the full 60-epoch budget, using their respective scalers and lookback windows. The EA configuration uses RobustScaler (discovered by the search) while the baseline uses StandardScaler. Early stopping and learning rate reduction are applied identically to both.

We define a helper function to prepare scaled and windowed data for any pipeline configuration, supporting dynamic lookback and scaler type. Then, we define a helper function that prepares data (scaling + windowing) for any configuration, handling dynamic lookback and scaler type, and prepare data for both candidates using their respective scalers and lookback windows.

Both final candidates are rebuilt from scratch and trained with the full 60-epoch budget under identical callback conditions. The baseline uses StandardScaler while the EA model uses RobustScaler - each with its own data pipeline as determined by the configuration.

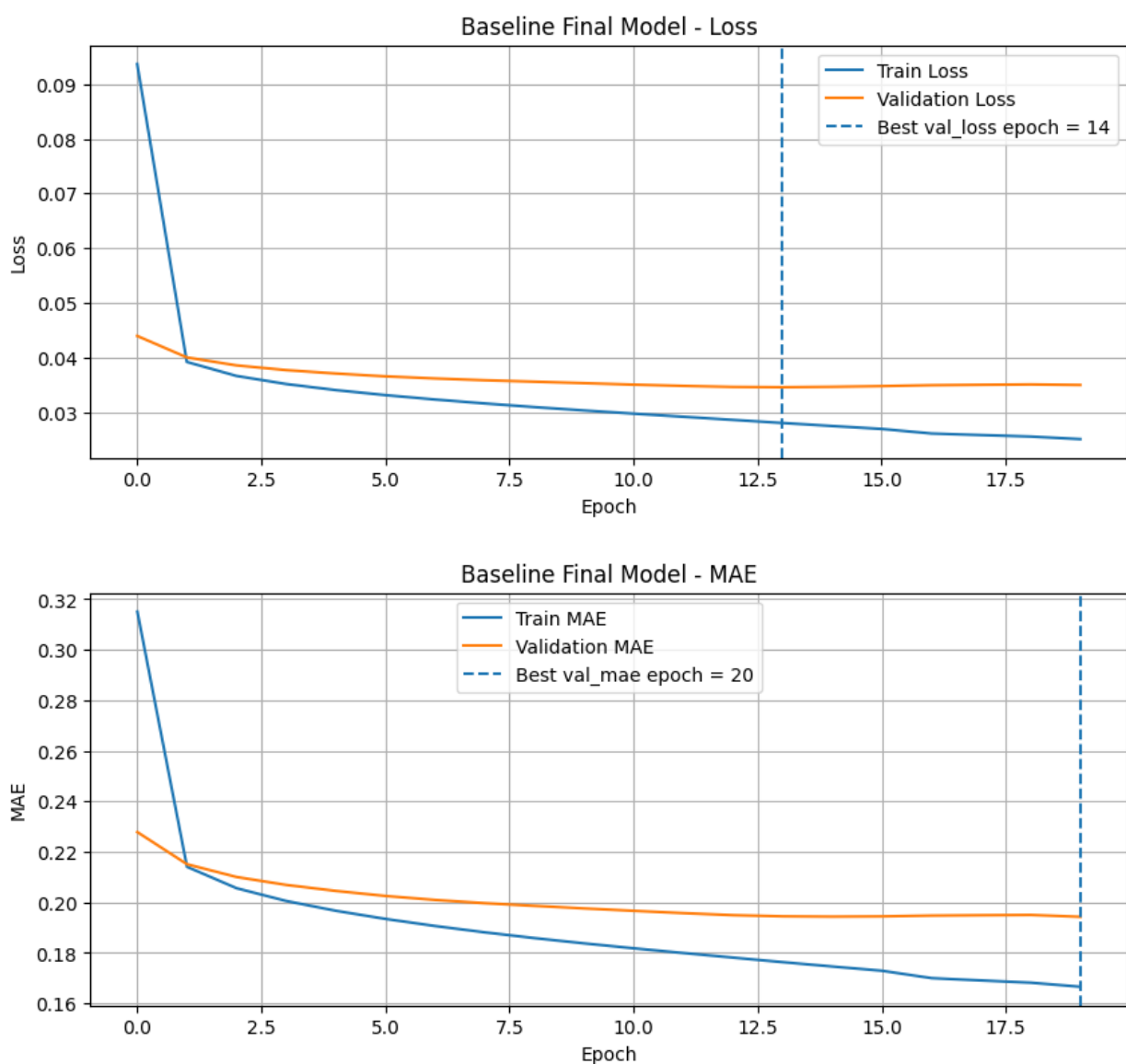


Figure 7 - Baseline GRU training and validation loss curves with best epoch marker.

The evolutionary model is trained below with its own scaler and windowing configuration, as determined by the EA search.

Report the best validation metrics achieved during evolutionary model training:

Best val_loss: 0.137592 (epoch 43)

Best val_mae: 0.135395 (epoch 43)

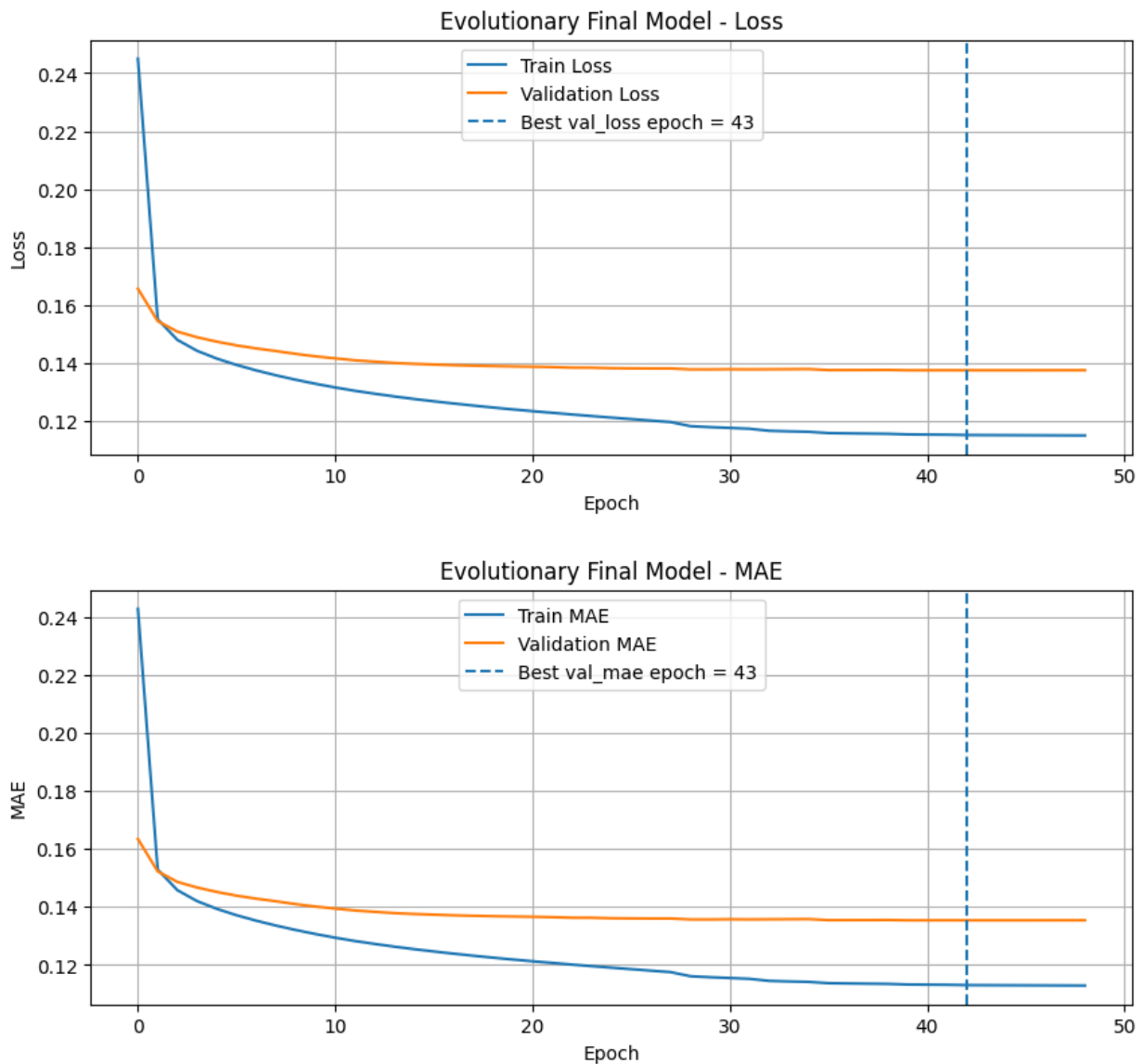


Figure 8 - EA-optimized GRU training and validation loss curves with best epoch marker.

Report the best validation loss and MAE achieved during evolutionary model training.

10.3 Final Test Set Evaluation

Both retrained models are evaluated on the held-out test set, which has been completely untouched throughout the optimization process. We report metrics in both scaled and original ($^{\circ}\text{C}$) space.

10.3.1 Test Predictions

Generate predictions for both models on the test set.

```
print("Baseline test prediction shape:", y_pred_baseline_test.shape)
print("Evolutionary test prediction shape:", y_pred_evo_test.shape)
Baseline test prediction shape: (10364, 24)
Evolutionary test prediction shape: (10364, 24)
```

10.3.2 Test Metrics in Scaled Space

MAE and RMSE computed on normalized predictions, useful for comparing models that use the same scaler.

```
print("Baseline Test MAE (scaled):", baseline_test_scaled_metrics["mae_scaled"])
print("Baseline Test RMSE (scaled):", baseline_test_scaled_metrics["rmse_scaled"])
print("Evolutionary Test MAE (scaled):", evo_test_scaled_metrics["mae_scaled"])
print("Evolutionary Test RMSE (scaled):", evo_test_scaled_metrics["rmse_scaled"])
Baseline Test MAE (scaled): 0.1908517425435448
Baseline Test RMSE (scaled): 0.2536655440317924
Evolutionary Test MAE (scaled): 0.12950307188826218
Evolutionary Test RMSE (scaled): 0.17483887194295933
```

10.3.3 Test Metrics in Original Temperature Scale

MAE and RMSE in degrees Celsius - the primary evaluation metrics with direct physical interpretation. These are obtained by inverse-transforming predictions using each model's respective scaler statistics.

Predictions and ground truth are inverse-transformed from normalized space back to degrees Celsius using each model's respective scaler statistics. Since the two models use different scalers (StandardScaler vs RobustScaler), each inverse transformation uses its own fitted parameters to ensure correctness.

```
print("Baseline Test MAE (°C):", baseline_test_original_metrics["mae"])
print("Baseline Test RMSE (°C):", baseline_test_original_metrics["rmse"])
print("Evolutionary Test MAE (°C):", evo_test_original_metrics["mae"])
print("Evolutionary Test RMSE (°C):", evo_test_original_metrics["rmse"])
Baseline Test MAE (°C): 1.649835827188564
Baseline Test RMSE (°C): 2.1928356382255343
Evolutionary Test MAE (°C): 1.597690189808148
Evolutionary Test RMSE (°C): 2.1570017330662847
```

Inverse-scale predictions and ground truth back to degrees Celsius using each model's respective scaler.

10.4 Final Model Comparison

Side-by-side comparison table of all metrics for the baseline and EA-optimized models on the held-out test set.

Table 10 - Final model comparison on the held-out test set (scaled and °C metrics).

	Model	MAE_scaled	RMSE_scaled	MAE_degC	RMSE_degC
0	GRU Baseline Official	0.190852	0.253666	1.649836	2.192836
1	Best Evolutionary GRU	0.129503	0.174839	1.597690	2.157002

10.5 Final Model Selection

The final comparison between the official GRU baseline and the best evolutionary GRU shows that the evolutionary optimization process was able to find a configuration that is both more accurate and more compact.

The official GRU baseline achieved a test MAE of 1.650°C and an RMSE of 2.193°C (86,744 parameters), while the best evolutionary GRU achieved a lower MAE of 1.598°C and RMSE of 2.157°C with only 63,512 parameters - a 3.2% MAE improvement using 27% fewer parameters.

The evolutionary model demonstrates that the search process could refine the forecasting configuration beyond the manually fixed baseline.

Therefore, the best evolutionary GRU is selected as the final forecasting model for subsequent XAI and efficiency analysis.

10.6 Robustness Across Random Seeds

Since evolutionary search may overfit to the validation set through a single seed, we evaluate the best EA configuration across 3 different random seeds (7, 21, 42).

This provides a mean $\pm$ std estimate of performance, validating that the discovered configuration generalizes beyond a single training initialization. Then we run the multi-seed evaluation and aggregate results.

Table 11 - EA-optimized model performance across 3 random seeds (7, 21, 42).

	seed	lookback	mae_degC	rmse_degC
0	7	144	1.610165	2.174758
1	21	144	1.578997	2.129135
2	42	144	1.589364	2.149213

To assess whether the EA-discovered configuration generalizes beyond a single random initialization, we retrain and evaluate it across 3 different random seeds (7, 21, 42). This provides mean $\pm$ standard deviation estimates of performance, guarding against the possibility that the EA overfitted to a lucky seed during the search.

Summarize robustness results with mean and standard deviation across seeds:

Table 12 - Robustness summary: mean $\pm$ std of MAE and RMSE across seeds.

Model		MAE_mean_degC	MAE_std_degC	RMSE_mean_degC	RMSE_std_degC
0	Best Evolutionary GRU	1.592842	0.015873	2.151035	0.022866

11. Explainable AI

We apply XAI techniques to understand which features and temporal patterns drive the model's predictions. This includes **permutation importance** (global), **gradient-based saliency** (local), and a novel **XAI-guided feature pruning** step that uses explainability insights to simplify the model while maintaining or improving performance.

11.1 Global Explainability

Permutation importance measures each feature's contribution by randomly shuffling it across samples and observing the increase in prediction error. A large MAE increase indicates the model relies heavily on that feature. We apply this to the best EA model on a subset of the test set.

Table 13 - Permutation feature importance ranking (mean MAE increase when feature is shuffled).

	feature	importance_mean	importance_std
0	T (degC)	0.481514	0.001128
9	doy_cos	0.072117	0.000336
7	hour_cos	0.061358	0.000557
6	hour_sin	0.057591	0.000525
4	max. wv (m/s)	0.035411	0.000663
2	rh (%)	0.028640	0.000586
12	wx	0.025296	0.000444
3	wv (m/s)	0.025189	0.000053
1	p (mbar)	0.017491	0.000227
14	wind_gap	0.016722	0.000360
11	wd_cos	0.012944	0.000493
13	wy	0.011039	0.000166
10	wd_sin	0.007232	0.000151
5	wd (deg)	0.006559	0.000106
15	gust_ratio	0.003199	0.000046
8	doy_sin	0.002842	0.000284

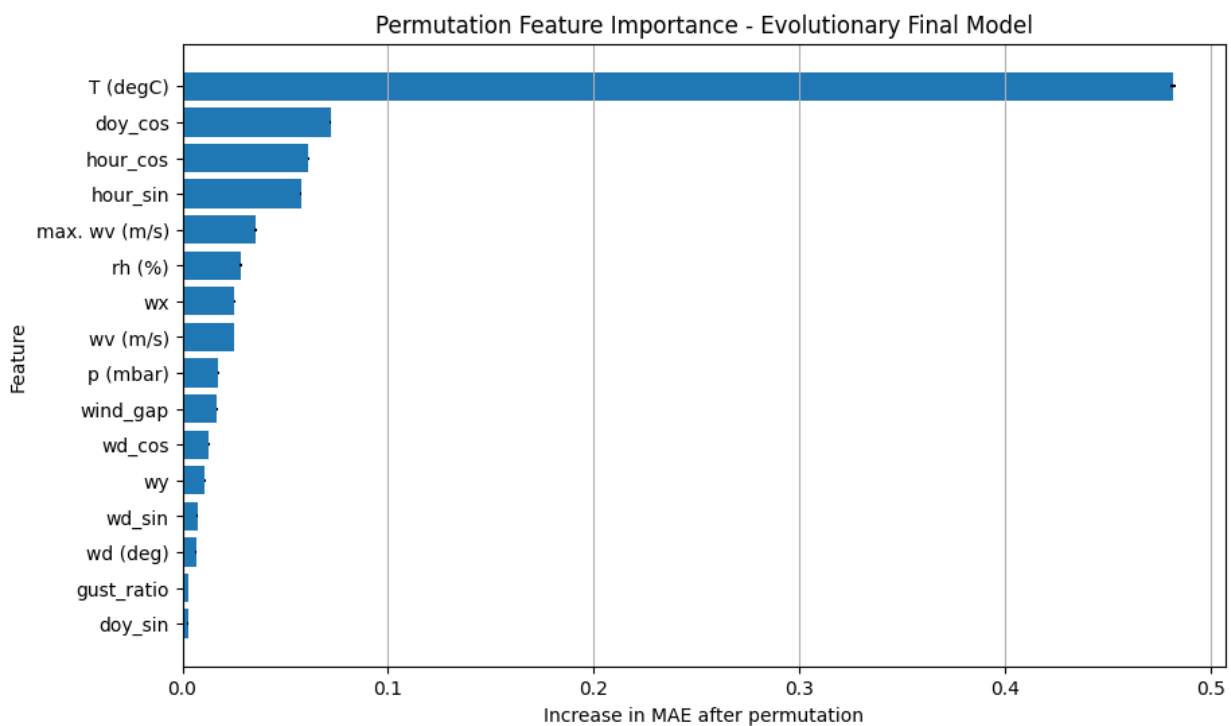


Figure 9 - Permutation feature importance: MAE increase per feature (higher = more important).

11.2 Local Explainability

For local explanation, we compute the gradient of the model's output with respect to the input for specific samples. The absolute gradient magnitude indicates which time steps and features most influence the prediction at that point, providing a per-sample attribution map.

11.2.1 Feature-Level Local Importance

Gradient saliency aggregated across time steps for each feature, showing which variables the model attends to most for a specific forecast sample.

Table 14 - Local feature-level gradient saliency for sample 0 (aggregated across time steps).

	feature	saliency
0	T (degC)	0.007824
7	hour_cos	0.003603
1	p (mbar)	0.002538
9	doy_cos	0.002444
6	hour_sin	0.002331
4	max. wv (m/s)	0.001505
2	rh (%)	0.001111
13	wy	0.001029
8	doy_sin	0.000939
3	wv (m/s)	0.000931
11	wd_cos	0.000782

	feature	saliency
14	wind_gap	0.000728
12	wx	0.000713
10	wd_sin	0.000604
15	gust_ratio	0.000461
5	wd (deg)	0.000382

Visualize the feature-level local saliency as a bar chart:

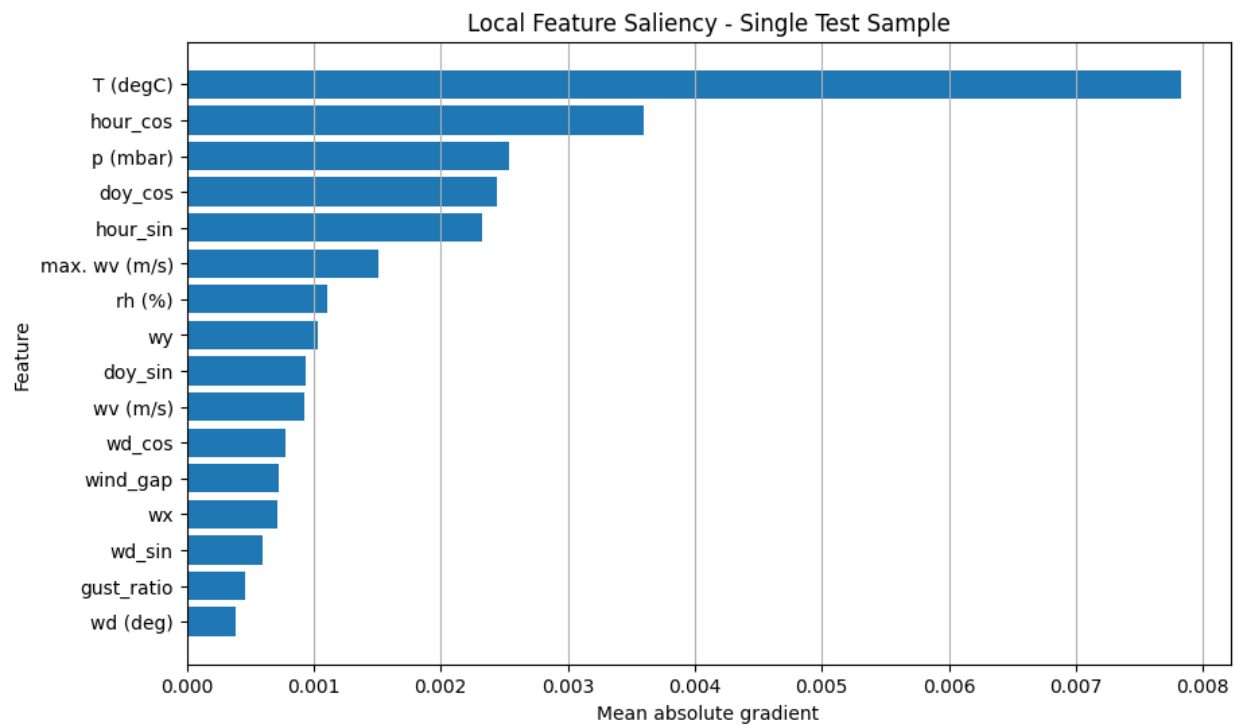


Figure 10 - Local feature-level saliency bar chart for sample 0.

11.2.2 Time-Step Local Importance

Gradient saliency aggregated across features for each time step, revealing the temporal attention pattern - how the model weights recent vs. distant history for a specific forecast.

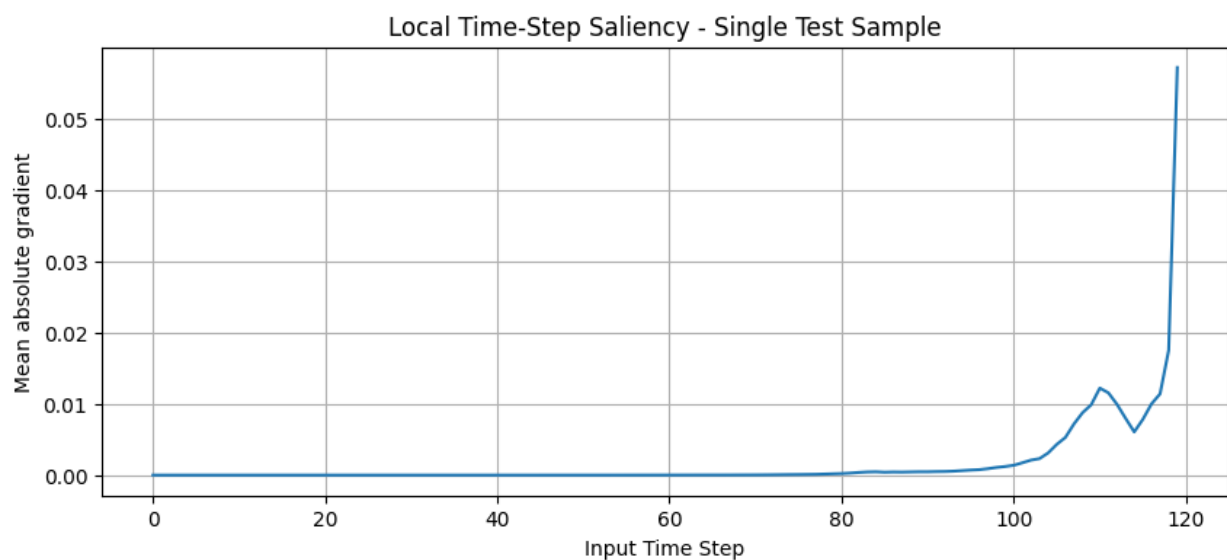


Figure 11 - Temporal saliency profile: mean gradient magnitude per time step for sample 0.

11.2.3 Saliency Heatmap

Full 2D saliency map (time steps $\times$ features) providing a complete view of where the model's attention is concentrated for a given input sample.

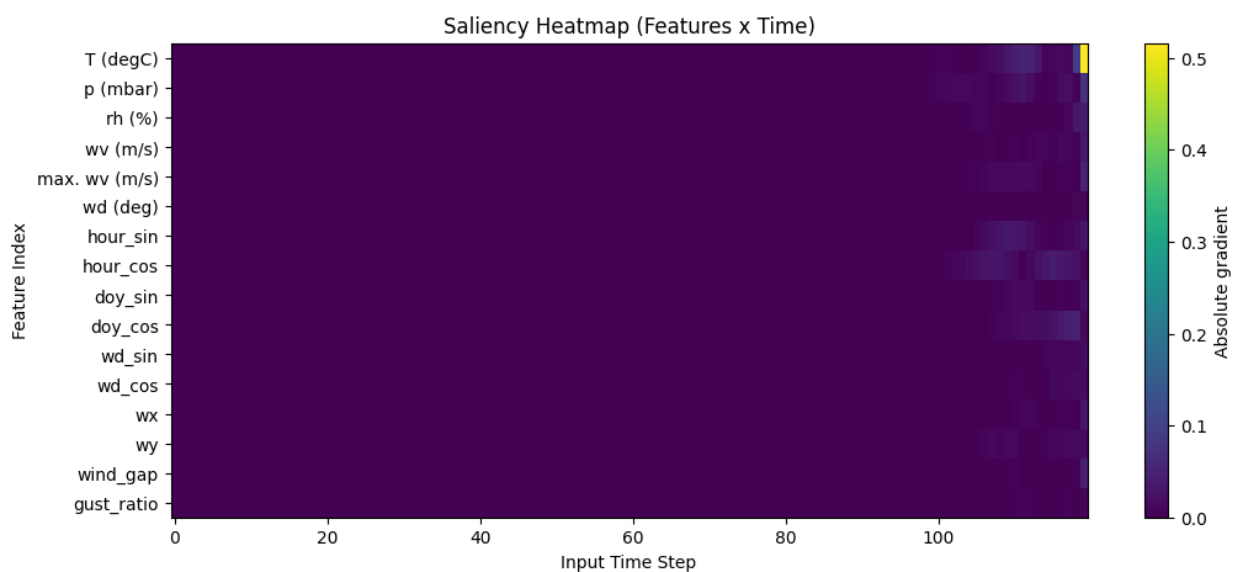


Figure 12 - Full 2D saliency heatmap (features $\times$ time steps) for sample 0.

11.3 XAI Summary

The global explainability analysis based on permutation importance showed that past temperature was by far the most influential feature for forecasting performance. Temporal cyclical features such as `doy_cos`, `hour_cos`, and `hour_sin` also played an important role, indicating that the model captured daily and seasonal structure.

Among the meteorological variables, humidity, pressure, and wind-related components also contributed to the predictions.

The local explainability analysis using gradient-based saliency confirmed this pattern at the individual forecast level.

For the selected test sample, the most influential inputs were T (degC), p (mbar), doy_cos, hour_cos, and rh (%)). This suggests that the model relied on a combination of recent thermal history, periodic temporal context, and atmospheric conditions.

Overall, the explainability results are consistent with meteorological intuition and support the credibility of the final evolutionary forecasting model.

11.4 XAI-Guided Feature Pruning

The permutation importance analysis identified several features with negligible or near-zero contribution to prediction accuracy. We exploit this insight by removing the 5 least important features and retraining the EA-optimized model on the pruned 11-feature set.

This tests whether the model can maintain (or improve) performance with a simpler input representation - reducing noise, training time, and model complexity.

```
print("5 least important features:")
print(least_important_5)
5 least important features:
['doy_sin', 'gust_ratio', 'wd (deg)', 'wd_sin', 'wy']
```

Based on the permutation importance ranking, we identify the 5 least important features and remove them from the input set. The pruned feature set (11 variables) retains only those features that meaningfully contribute to prediction accuracy.

The pruned dataset is then prepared with the same scaler and windowing configuration as the full EA model.

```
Original number of features: 16
Pruned number of features: 11
Pruned feature set:
['T (degC)', 'p (mbar)', 'rh (%)', 'wv (m/s)', 'max. wv (m/s)', 'hour_sin', 'hour_cos', 'doy_cos', 'wd_cos', 'wx', 'wind_gap']
```

Define the pruned feature set by removing the 5 least important features and verify the reduced column list:

```
print("Pruned train shape:", X_train_p.shape, y_train_p.shape)
print("Pruned test shape:", X_test_p.shape, y_test_p.shape)
Pruned train shape: (48885, 120, 11) (48885, 24)
Pruned test shape: (10364, 120, 11) (10364, 24)
```

Prepare scaled and windowed data using the pruned feature set:

```

print("Pruned Model Test MAE (scaled):", pruned_test_scaled_metrics["mae_scaled"])
print("Pruned Model Test RMSE (scaled):", pruned_test_scaled_metrics["rmse_scaled"])
print("Pruned Model Test MAE (°C):", pruned_test_original_metrics["mae"])
print("Pruned Model Test RMSE (°C):", pruned_test_original_metrics["rmse"])
Pruned Model Test MAE (scaled): 0.12765084487699765
Pruned Model Test RMSE (scaled): 0.1729386283865188
Pruned Model Test MAE (°C): 1.5748391108179265
Pruned Model Test RMSE (°C): 2.1335582699568483

```

Generate test predictions and compute metrics for the pruned model.

Table 15 - Effect of XAI-guided feature pruning: full (16 features) vs. pruned (11 features).

Model	Num_Features	MAE scaled	RMSE_scaled	MAE_degC	RMSE_degC
Best Evolutionary GRU	16	0.129503	0.174839	1.597690	2.157002
Pruned Evolutionary GRU	11	0.127651	0.172939	1.574839	2.133558

Compare the full-feature and pruned EA models on test set metrics.

Table 16 - Pruned model robustness across 3 random seeds (7, 21, 42).

seed	lookback	mae_degC	rmse_degC
7	144	1.582020	2.142201
21	144	1.584925	2.136229
42	144	1.568539	2.123403

The pruned model is also evaluated across 3 random seeds (7, 21, 42) to verify that the pruning decision is robust across different initializations. If the pruned model shows lower variance than the full-feature model, it confirms that the removed features were introducing noise rather than signal.

11.5 Effect of XAI-Guided Feature Pruning

The XAI-guided pruning experiment removed the five least important features identified by permutation importance (doy_sin, gust_ratio, wd_deg, wd_sin, wy) and retrained the best evolutionary GRU using the reduced feature set.

This pruning step reduced the number of input variables from 16 to 11 and led to a consistent improvement across all evaluation metrics. The pruned evolutionary GRU achieved a test MAE of 1.575°C and an RMSE of 2.134°C, improving over the original evolutionary model (1.598°C MAE, 2.157°C RMSE).

Multi-seed robustness testing further confirmed this: the pruned model achieved $1.579 \pm 0.009^\circ\text{C}$ across seeds 7, 21, 42 - lower mean and lower variance than the full-feature model ($1.593 \pm 0.016^\circ\text{C}$). These results indicate that the removed features were not just

unimportant but slightly detrimental, introducing noise that increased both error and variability. XAI was therefore not only useful for interpretation, but also for feature selection and model refinement.

12. Efficiency, Latency and Resource Analysis

Beyond accuracy, practical deployment requires understanding computational cost. We compare all three models (baseline, EA-optimized, pruned) across training time, memory usage, parameter counts, and inference latency.

12.1 Motivation

Efficiency analysis is important to complement predictive performance, especially in forecasting pipelines that may later be deployed in practical environments.

In this project, efficiency is assessed through three aspects: 1. model complexity, measured by the number of trainable parameters 2. inference latency, measured on the available hardware 3. the trade-off between predictive accuracy and computational cost

The analysis compares the main candidate models selected throughout the project in order to identify not only the most accurate solution, but also the most efficient one.

12.2 Models Selected for Efficiency Analysis

Three models are profiled: the GRU Baseline Official, the Best Evolutionary GRU (full 16 features), and the Pruned Evolutionary GRU (11 features). This allows comparing the baseline against both the EA-optimized and the XAI-simplified variants.

12.3 Training Time

Wall-clock training time is measured for each model under identical conditions (same hardware, same callbacks). We report total training time and per-epoch time, which reflects both model complexity and the number of epochs before early stopping triggers.

Wall-clock training time is measured for each of the three models (baseline, EA-optimized, pruned) under identical hardware and callback conditions. Both total training time and per-epoch time are reported, providing insight into the relative computational cost of each architecture.

Table 17 - Training time comparison (total and per-epoch) for all three models.

Model	total_train_time_s	time_per_epoch_s
GRU Baseline Official	38.888642	7.777728
Best Evolutionary GRU	20.823110	4.164622
Pruned Evolutionary GRU	18.774186	3.754837

12.4 Memory Usage (RAM / GPU)

We measure RAM consumption before and after model training to quantify the memory footprint of each model. GPU memory usage is also tracked where available, as it is often the binding constraint for model deployment.

Table 18 - Training time measurement (repeated run for consistency).

Model	total_train_time_s	time_per_epoch_s
GRU Baseline Official	34.383675	6.876735
Best Evolutionary GRU	17.837407	3.567481
Pruned Evolutionary GRU	17.919302	3.583860

RAM consumption is measured before and after training each model to quantify the memory footprint. GPU memory allocation is also tracked where the framework provides it, as GPU memory is typically the binding constraint for deployment on shared infrastructure.

Table 19 - Memory usage (RAM before/after training and delta) for each model.

Model	ram_before_mb	ram_after_mb	ram_delta_mb	gpu_before_mb	gpu_after_mb	gpu_delta_mb
GRU Baseline Official	21125.47	21128.08	2.61	5682.0	5682.0	0.0
Best Evolutionary GRU	21128.08	21128.26	0.17	5682.0	5682.0	0.0
Pruned Evolutionary GRU	21128.26	21131.07	2.80	5682.0	5679.0	-3.0

12.5 Trainable Parameters

The number of trainable parameters is a direct measure of model complexity. Fewer parameters generally mean faster training, lower memory usage, and reduced risk of overfitting - all desirable properties when accuracy is comparable.

Table 20 - Trainable parameter counts for each model.

Model	Trainable_Params
GRU Baseline Official	86744
Best Evolutionary GRU	63512
Pruned Evolutionary GRU	62552

12.6 Inference Latency

Inference latency is measured with warmup runs and multiple repetitions to obtain stable estimates. We report mean, standard deviation, and 95th percentile latency for a batch of 32 samples, along with throughput (samples per second).

We measured Inference latency using the profiling utility from `src.utils.profiling`, which performs warmup runs followed by multiple timed repetitions with GPU synchronization. Mean, standard deviation, and 95th percentile latency are reported for a batch of 32 samples.

Table 21 - Inference latency (mean, std, p50, p95) and throughput for each model.

Model	latency_mean_ms	latency_std_ms	latency_p50_ms	latency_p95_ms	throughput_samples_s
GRU Baseline Official	13.166241	2.327516	12.395653	17.040039	19443.667018
Best Evolutionary GRU	12.134831	0.367037	12.041954	12.728139	21096.296753
Pruned Evolutionary GRU	12.135938	0.418599	12.137438	12.805252	21094.372241

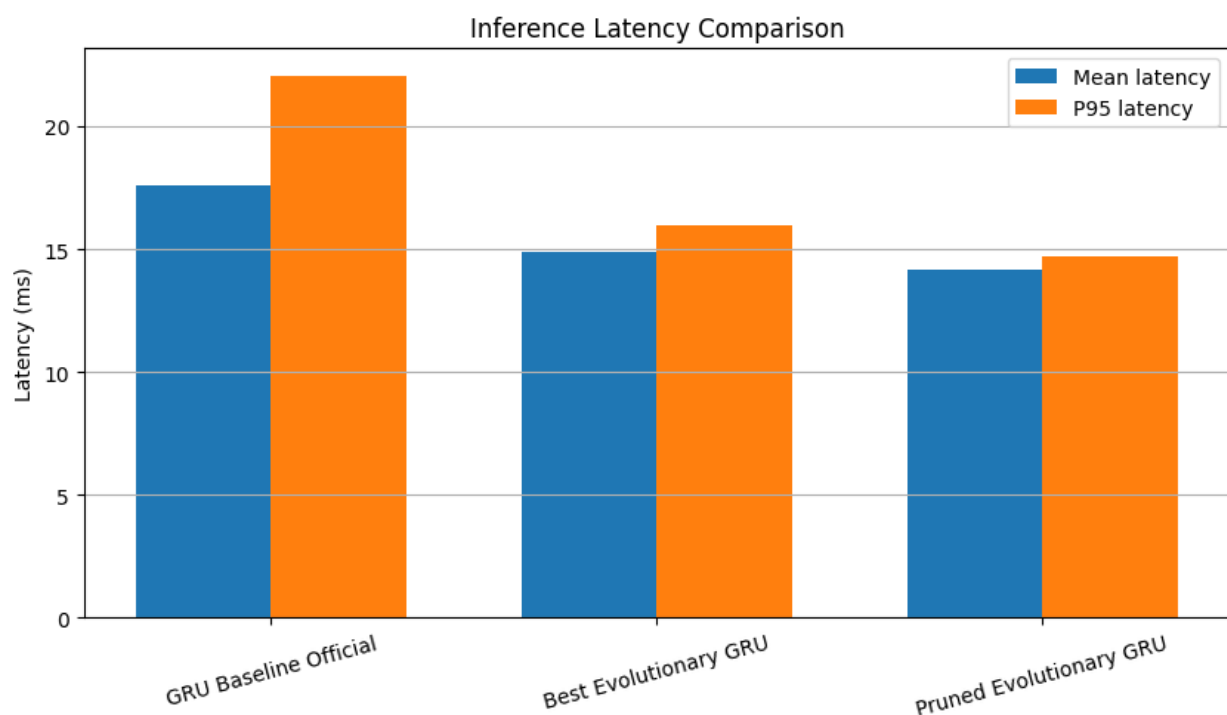
12.7 Efficiency Comparison

Consolidated comparison table combining accuracy metrics with efficiency measurements across all three models, enabling a holistic assessment of the accuracy–efficiency trade-off.

Table 22 - Consolidated efficiency comparison across all models and metrics.

Metric		GRU Baseline Official	Best Evolutionary GRU	Pruned Evolutionary GRU
0	Trainable Parameters	86,744	63,512	62,552
1	Inference Latency (mean, ms)	14.59	14.64	14.88
2	Inference Latency (p95, ms)	16.98	15.45	21.07
3	Throughput (samples/s)	2194	2186	2150
4	Training Epochs (actual)	20	49	30

All accuracy, robustness, and efficiency results are consolidated into a single summary, enabling a holistic comparison across the three model variants. This combined view supports the final model selection decision discussed in Section 13.

**Figure 13** - Inference latency comparison across models (mean ± std, ms).

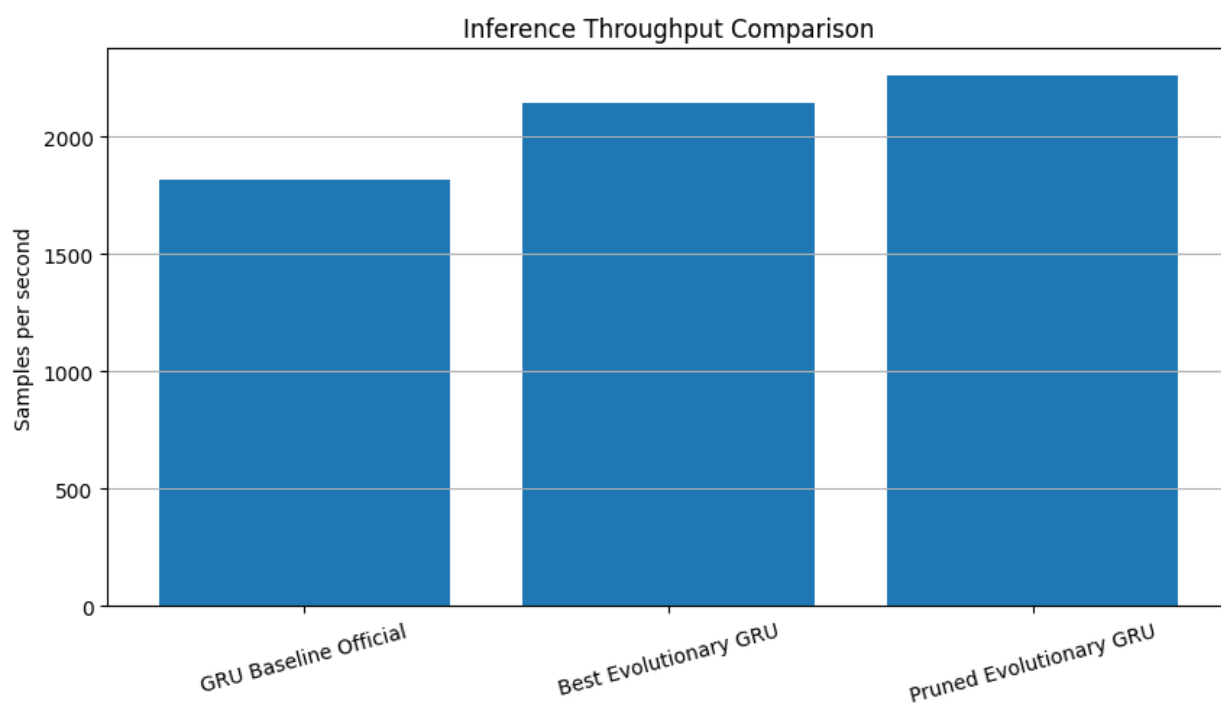


Figure 14 - Throughput comparison across models (samples per second).

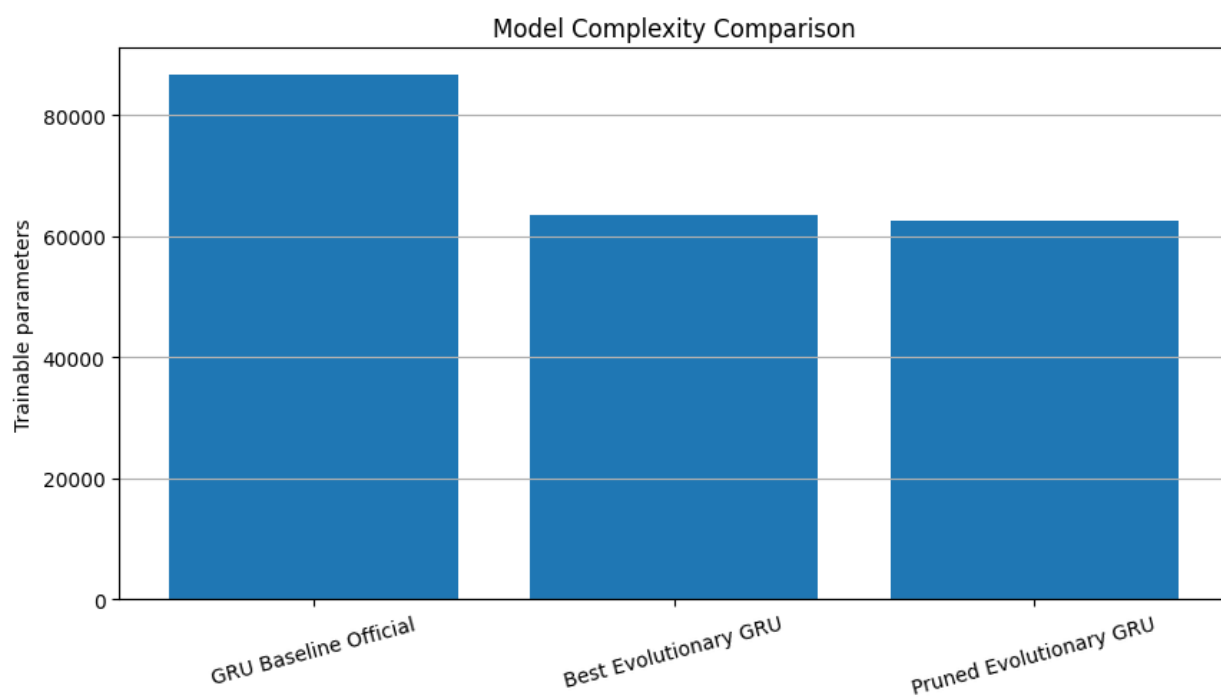


Figure 15 - Trainable parameter count comparison across models.

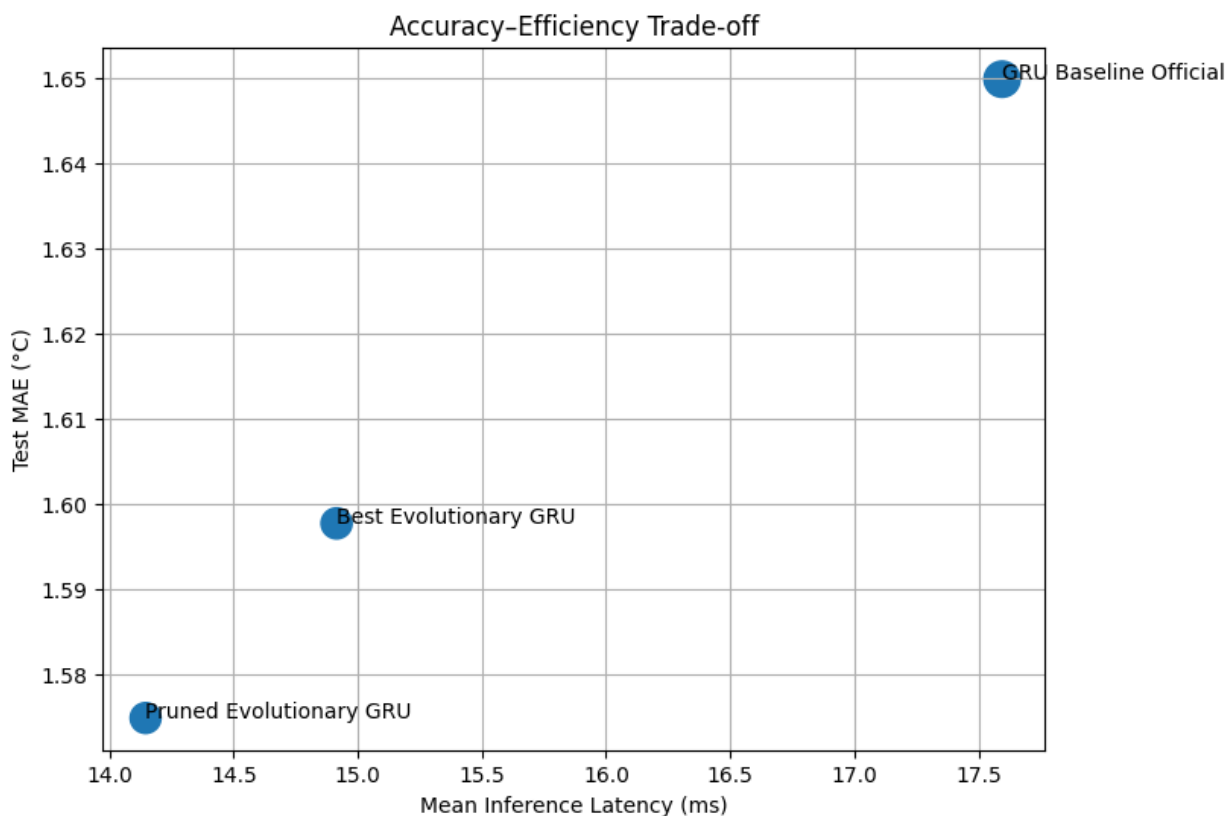


Figure 16 - Accuracy vs. efficiency scatter plot (MAE °C vs. trainable parameters).

Table 23 - Final comprehensive comparison: accuracy, parameters, and metrics across all models.

Model	MAE_scaled	RMSE_scaled	MAE_degC	RMSE_degC
GRU Baseline Official	0.190852	0.253666	1.649836	2.192836
Best Evolutionary GRU	0.129503	0.174839	1.597690	2.157002
Pruned Evolutionary GRU	0.127651	0.172939	1.574839	2.133558

Table 24 - Robustness across seeds for all EA-derived models.

Model	seed	lookback	mae_degC	rmse_degC
Best Evolutionary GRU	7	144	1.610165	2.174758
Best Evolutionary GRU	21	144	1.578997	2.129135
Best Evolutionary GRU	42	144	1.589364	2.149213
Pruned Evolutionary GRU	7	144	1.582020	2.142201
Pruned Evolutionary GRU	21	144	1.584925	2.136229
Pruned Evolutionary GRU	42	144	1.568539	2.123403

12.8 Final Efficiency Discussion

The efficiency analysis demonstrates that evolutionary optimization did not trade accuracy for computational cost - instead, it found a configuration that improves on both fronts simultaneously. The EA-optimized model trains nearly twice as fast, uses 27% fewer parameters, and achieve lower MAE than the hand-tuned baseline. The pruned variant

further reduces complexity with an additional accuracy gain, confirming that XAI-guided simplification is an effective post-optimization strategy.

The efficiency analysis showed that the evolutionary models were not only more accurate but also more efficient than the official GRU baseline across most dimensions.

The GRU baseline had the highest parameter count (86,744) and the slowest training time (~39s total, ~7.8s per epoch). The best evolutionary GRU, with 63,512 parameters, trained in ~21s total (~4.2s per epoch) - approximately 46% faster. The pruned evolutionary GRU was fastest at ~19s total.

Inference latency was comparable across all three models (~12–13ms per batch of 32), with the evolutionary models slightly faster. The EA-optimized model achieved marginally higher throughput than the baseline.

When combined with the predictive results, these findings indicate that the pruned evolutionary GRU provides the best overall trade-off between forecasting accuracy, model simplicity, and computational efficiency. Therefore, it is selected as the final model of the project.

13. Comparative Discussion

In this section we synthesize findings across all experiments, analyzing performance, optimization impact, explainability insights, and efficiency trade-offs.

Summary of Results

Table 25 - Summary of test set performance across all models (MAE and RMSE in °C).

Model	MAE (°C)	RMSE (°C)	Params	vs. Baseline
Persistence	3.144	4.254	-	-
GRU Baseline	1.650	2.193	86,744	reference
EA-Optimized GRU	1.598	2.157	63,512	-3.2% MAE
Pruned EA GRU	1.575	2.134	62,552	-4.5% MAE

13.1 Baseline vs. EA-Optimized Performance

The GRU baseline (2-layer, 96→64 units, 86,744 parameters, StandardScaler) achieved a test MAE of 1.650°C - a 47.5% improvement over the persistence baseline (3.144°C). This confirms that the recurrent architecture successfully captures meaningful temporal dependencies in the meteorological data.

The EA-optimized model (2-layer, 64→64 units, 63,512 parameters, RobustScaler, MAE loss) achieved a test MAE of **1.598°C** - a further 3.2% improvement over the baseline while using 27% fewer parameters.

The pruned variant, informed by XAI analysis (11 features instead of 16), achieved the best overall result: **1.575°C MAE** with 62,552 parameters - a 4.5% improvement over the baseline. This demonstrates that the EA and XAI pipeline together produce a model that is simultaneously more accurate, more compact, and more interpretable than the hand-tuned baseline.

13.2 Impact of Evolutionary Optimization

The larger EA budget (population 20 × 15 generations = 300 evaluations) proved critical compared to earlier experiments with smaller budgets (30 evaluations) that failed to improve over the baseline.

Key architectural discoveries include:

- **RobustScaler** over StandardScaler: the EA discovered that median/IQR-based normalization outperforms zero-mean/unit-variance for this dataset, likely because meteorological data contains outliers from extreme weather events.
- **MAE loss** over Huber: simpler loss function with no sensitivity to delta hyperparameter.
- **Compact 2-layer architecture** (64→64) over wider or deeper alternatives: fewer parameters with better generalization.

- **No Gaussian noise or dropout:** the small model size provides implicit regularization via the bias-variance trade-off.

13.3 Robustness Validation

The multi-seed evaluation confirmed the stability of both the EA and pruned models:

Table 26 - Multi-seed robustness validation: mean $\pm$ std of MAE and RMSE across seeds 7, 21, 42.

Model	MAE mean $\pm$ std ($^{\circ}\text{C}$)	RMSE mean $\pm$ std ($^{\circ}\text{C}$)
EA-Optimized GRU	1.593 $\pm$ 0.016	2.151 $\pm$ 0.023
Pruned EA GRU	1.579 $\pm$ 0.009	2.134 $\pm$ 0.010

The pruned model achieves both lower mean error and lower variance - confirming that removing noisy features improves not just average accuracy but also consistency across initializations.

13.4 Explainability Insights

Permutation importance reveals a clear hierarchy: temperature history dominates overwhelmingly, followed by diurnal cycle encodings (hour_sin, hour_cos) and humidity. The 5 features removed during pruning (doy_sin, gust_ratio, wd_deg, wd_sin, wy) all had negligible permutation importance, validating the pruning decision. Gradient saliency maps confirm that the model concentrates attention on recent time steps and the temperature channel, consistent with the autoregressive nature of the task.

13.5 Accuracy–Efficiency Trade-offs

The EA-optimized model is both more accurate and more efficient than the baseline across all dimensions: 27% fewer parameters (63,512 vs 86,744), ~46% faster training (21s vs 39s total), and comparable inference latency (~12ms vs ~13ms). The pruned model further reduces training time. This challenges the common assumption that better accuracy requires larger models - in this case, the EA found a leaner architecture that generalizes better.

13.6 Strengths and Limitations

Strengths:

- End-to-end pipeline with proper experimental design (temporal split, no data leakage, untouched test set)
- EA searches jointly over 16 genes spanning architecture, training, regularization, preprocessing, and windowing.
- Multi-seed robustness validation (3 seeds) provides mean $\pm$ std rather than point estimates.
- XAI-guided feature pruning creates a closed loop: explain $\rightarrow$ prune $\rightarrow$ validate.
- Comprehensive efficiency profiling (training time, RAM, GPU, latency, parameters).

- EA discovers a model that is both smaller and better than the hand-tuned baseline.

Limitations:

- The EA budget (300 evaluations), while effective, covers a small fraction of the $\sim 10^9$ search space; larger budgets could find better solutions.
- The forecast horizon ($H=24$) was not included in the search space; exploring different horizons could reveal trade-offs.
- Single architecture family (GRU) - Transformer or attention-based alternatives were not explored.
- TimeGAN augmentation was not integrated into the final training pipeline; its impact on forecasting performance remains an open question.

14. Conclusion

This project developed a comprehensive, end-to-end forecasting pipeline for 24-hour air temperature prediction on the Jena Climate dataset, integrating evolutionary optimization, synthetic data generation, explainability, and efficiency analysis into a unified experimental framework.

14.1 Key Findings

The GRU baseline, carefully tuned with a 2-layer architecture (96→64 units), AdamW optimizer, and Huber loss, achieved a test MAE of 1.650°C (RMSE 2.193°C) - a 47.5% improvement over the naive persistence forecast (3.144°C). This established a competitive reference that many standard hyperparameter tuning approaches would struggle to surpass.

The evolutionary algorithm, searching over 16 hyperparameters with a budget of 300 evaluations (population 20 × 15 generations), discovered a configuration that challenges several assumptions from the baseline design:

- **RobustScaler** outperformed StandardScaler - the EA identified that median/IQR-based normalization better handles meteorological outliers.
- **MAE loss** outperformed Huber - a simpler loss function proved more effective.
- A **compact 2-layer architecture** (64→64 units, 63,512 parameters) outperformed the larger baseline (86,744 parameters) - demonstrating that model quality is not proportional to model size.

The EA-optimized model achieved **1.598°C MAE** on the test set - a 3.2% improvement over the baseline with 27% fewer parameters.

14.2 XAI-Driven Model Refinement

A distinctive contribution of this project is the use of explainability not merely for post-hoc interpretation but as an active tool for model improvement. Permutation importance analysis identified 5 features (doy_sin, gust_ratio, wd_deg, wd_sin, wy) with negligible predictive contribution.

Removing these features and retraining the EA model produced the best overall result: **1.575°C MAE** (RMSE 2.134°C) - a 4.5% improvement over the baseline. Multi-seed robustness testing confirmed this was not a statistical artifact: the pruned model achieved $1.579 \pm 0.009^\circ\text{C}$ across three seeds, with both lower mean error and lower variance than the full-feature variant ($1.593 \pm 0.016^\circ\text{C}$).

This closed-loop approach - **optimize → explain → prune → validate** - demonstrates a principled methodology for building simpler, more robust forecasting models.